

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**

— o0o —



BÁO CÁO CHUYÊN ĐỀ CÔNG NGHỆ PHẦN MỀM

**CÁC THUẬT TOÁN ĐỐI SÁNH MẪU
(PATTERN MATCHING ALGORITHMS)**

Họ và tên : Nguyễn Hoàng Tùng
Mã sinh viên : B22DCDT294
Lớp : E22CNPM03
Hệ đào tạo : Chính quy
Giảng viên hướng dẫn : TS. Nguyễn Duy Phương

Hà Nội - 2026

Mục lục

1	Tổng quan	7
2	Tìm kiếm mẫu từ trái sang phải	9
2.1	Thuật toán Brute Force	9
2.1.1	Trình bày thuật toán	9
2.1.2	Đánh giá độ phức tạp thuật toán	9
2.1.3	Kiểm nghiệm thuật toán	9
2.1.4	Lập trình theo thuật toán	13
2.2	Search with an automaton	14
2.2.1	Trình bày thuật toán	14
2.2.2	Đánh giá độ phức tạp thuật toán	14
2.2.3	Kiểm nghiệm thuật toán	14
2.2.4	Lập trình theo thuật toán	15
2.3	Thuật toán Karp-Rabin	16
2.3.1	Trình bày thuật toán	16
2.3.2	Đánh giá độ phức tạp thuật toán	16
2.3.3	Kiểm nghiệm thuật toán	17
2.3.4	Lập trình theo thuật toán	17
2.4	Thuật toán Shift Or	18
2.4.1	Trình bày thuật toán	18
2.4.2	Đánh giá độ phức tạp thuật toán	18
2.4.3	Kiểm nghiệm thuật toán	18

2.4.4	Lập trình theo thuật toán	19
2.5	Thuật toán Morris-Pratt	20
2.5.1	Trình bày thuật toán	20
2.5.2	Đánh giá độ phức tạp thuật toán	20
2.5.3	Kiểm nghiệm thuật toán	21
2.5.4	Lập trình theo thuật toán	23
2.6	Thuật toán Knuth-Morris-Pratt	24
2.6.1	Trình bày thuật toán	24
2.6.2	Đánh giá độ phức tạp thuật toán	24
2.6.3	Kiểm nghiệm thuật toán	24
2.6.4	Lập trình theo thuật toán	27
2.7	Thuật toán Apostolico-Crochemore	28
2.7.1	Trình bày thuật toán	28
2.7.2	Đánh giá độ phức tạp thuật toán	29
2.7.3	Kiểm nghiệm thuật toán	29
2.7.4	Lập trình theo thuật toán	31
2.8	Thuật toán Not So Naïve	32
2.8.1	Trình bày thuật toán	32
2.8.2	Đánh giá độ phức tạp thuật toán	33
2.8.3	Kiểm nghiệm thuật toán	33
2.8.4	Lập trình theo thuật toán	37
3	Tìm kiếm mẫu từ phải sang trái	38
3.1	Thuật toán Boyer-Moore	38
3.1.1	Trình bày thuật toán	38
3.1.2	Đánh giá độ phức tạp thuật toán	39
3.1.3	Kiểm nghiệm thuật toán	40
3.1.4	Lập trình theo thuật toán	42
3.2	Thuật toán Turbo-BM	43
3.2.1	Trình bày thuật toán	43

3.2.2	Đánh giá độ phức tạp thuật toán	45
3.2.3	Kiểm nghiệm thuật toán	45
3.2.4	Lập trình theo thuật toán	47
3.3	Thuật toán Quick Search	48
3.3.1	Trình bày thuật toán	48
3.3.2	Đánh giá độ phức tạp thuật toán	48
3.3.3	Kiểm nghiệm thuật toán	49
3.3.4	Lập trình theo thuật toán	51
3.4	Thuật toán Tuned Boyer-Moore	51
3.4.1	Trình bày thuật toán	51
3.4.2	Đánh giá độ phức tạp thuật toán	51
3.4.3	Kiểm nghiệm thuật toán	52
3.4.4	Lập trình theo thuật toán	54
3.5	Thuật toán Zhu-Takaoka	55
3.5.1	Trình bày thuật toán	55
3.5.2	Đánh giá độ phức tạp thuật toán	55
3.5.3	Kiểm nghiệm thuật toán	56
3.5.4	Lập trình theo thuật toán	57
3.6	Thuật toán Berry-Ravindran	58
3.6.1	Trình bày thuật toán	58
3.6.2	Đánh giá độ phức tạp thuật toán	59
3.6.3	Kiểm nghiệm thuật toán	59
3.6.4	Lập trình theo thuật toán	61
3.7	Thuật toán Apostolico-Giancarlo	62
3.7.1	Trình bày thuật toán	62
3.7.2	Đánh giá độ phức tạp thuật toán	64
3.7.3	Kiểm nghiệm thuật toán	64
3.7.4	Lập trình theo thuật toán	66
4	Tìm kiếm mẫu từ vị trí xác định	69

4.1	Thuật toán Colussi	69
4.1.1	Trình bày thuật toán	69
4.1.2	Đánh giá độ phức tạp thuật toán	70
4.1.3	Kiểm nghiệm thuật toán	70
4.1.4	Lập trình theo thuật toán	73
4.2	Thuật toán Skip Search	74
4.2.1	Trình bày thuật toán	74
4.2.2	Đánh giá độ phức tạp thuật toán	75
4.2.3	Kiểm nghiệm thuật toán	75
4.2.4	Lập trình theo thuật toán	76
4.3	Thuật toán Alpha Skip Search	77
4.3.1	Trình bày thuật toán	77
4.3.2	Đánh giá độ phức tạp thuật toán	77
4.3.3	Kiểm nghiệm thuật toán	77
4.3.4	Lập trình theo thuật toán	80
5	Tìm kiếm mẫu từ vị trí bất kỳ	81
5.1	Thuật toán Horspool	81
5.1.1	Trình bày thuật toán	81
5.1.2	Đánh giá độ phức tạp thuật toán	81
5.1.3	Kiểm nghiệm thuật toán	81
5.1.4	Lập trình theo thuật toán	84
5.2	Thuật toán Smith	85
5.2.1	Trình bày thuật toán	85
5.2.2	Đánh giá độ phức tạp thuật toán	85
5.2.3	Kiểm nghiệm thuật toán	85
5.2.4	Lập trình theo thuật toán	87
5.3	Thuật toán Raita	88
5.3.1	Trình bày thuật toán	88
5.3.2	Đánh giá độ phức tạp thuật toán	88

5.3.3	Kiểm nghiệm thuật toán	88
5.3.4	Lập trình theo thuật toán	91

Chương 1

Tổng quan

Các thuật toán tìm kiếm mẫu từ **trái sang phải** gồm:

- Thuật toán Brute Force
- Search with an automaton
- Thuật toán Karp-Rabin
- Thuật toán Shift Or
- Thuật toán Morris-Pratt
- Thuật toán Knuth-Morris-Pratt
- Thuật toán Apostolico-Crochemore
- Thuật toán Not So Naïve

Các thuật toán tìm kiếm mẫu từ **phải sang trái** gồm:

- Thuật toán Boyer-Moore
- Thuật toán Turbo-BM
- Thuật toán Quick Search
- Thuật toán Tuned-Boyer-Moore
- Thuật toán Zhu-Takaoka
- Thuật toán Berry-Ravindran

- Thuật toán Apostolico-Giancarlo

Các thuật toán tìm kiếm mẫu từ **vị trí xác định** gồm:

- Thuật toán Colussi
- Thuật toán Skip Search
- Thuật toán Alpha Skip Search

Các thuật toán tìm kiếm mẫu từ **vị trí bất kỳ** gồm:

- Thuật toán Horspool
- Thuật toán Smith
- Thuật toán Raita

Chương 2

Tìm kiếm mẫu từ trái sang phải

2.1 Thuật toán Brute Force

2.1.1 Trình bày thuật toán

Thuật toán Brute Force kiểm tra tất cả vị trí trong đoạn văn bản từ vị trí 0 đến $n - m$, xem liệu mẫu (pattern) có khớp ngay vị trí bắt đầu không. Sau mỗi lần kiểm tra, nó dịch mẫu sang phải 1 vị trí. Đặc điểm: không cần pha tiền xử lý, bộ nhớ cần dùng cố định.

2.1.2 Đánh giá độ phức tạp thuật toán

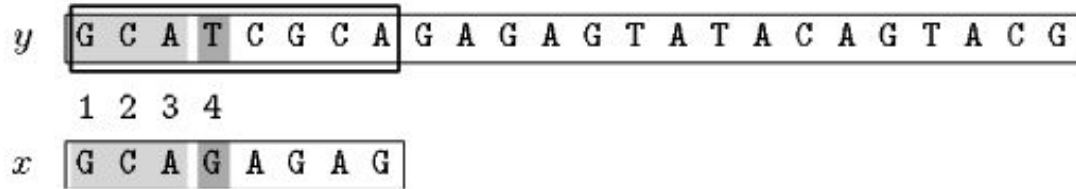
- Độ phức tạp pha thực thi là $O(m \times n)$.

2.1.3 Kiểm nghiệm thuật toán

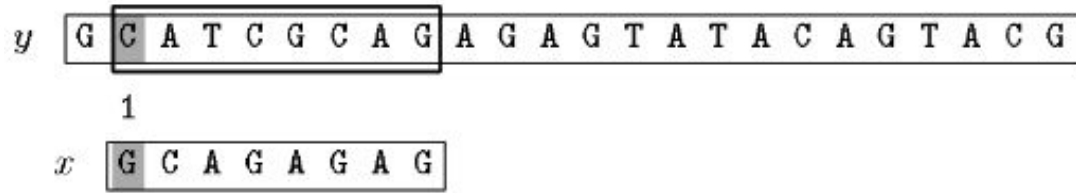
Giả sử ta có 2 mảng ký tự y và x . Ta tiến hành tìm mảng x trong mảng y với:

- $x = \text{GCAGAGAG}$, độ dài $m = 8$
- $y = \text{GCATCGCAGAGAGTATACAGTACG}$, độ dài $n = 24$

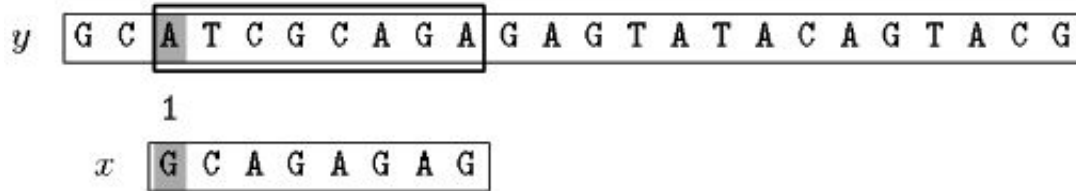
Pha tìm kiếm:



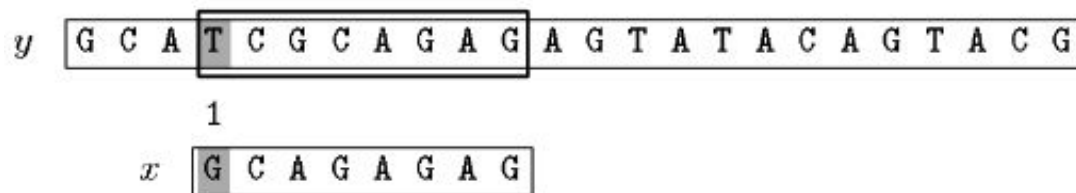
Hình 2.1: Lần 1: So sánh từ vị trí 0



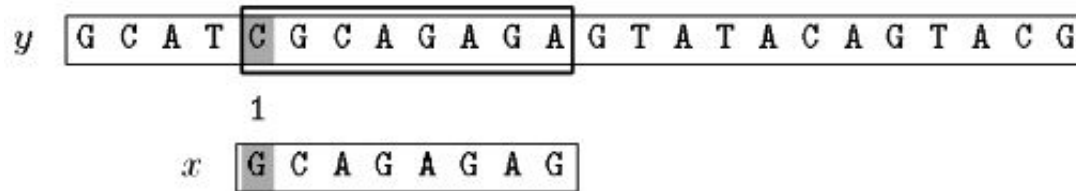
Hình 2.2: Lần 2: Dịch 1 bước



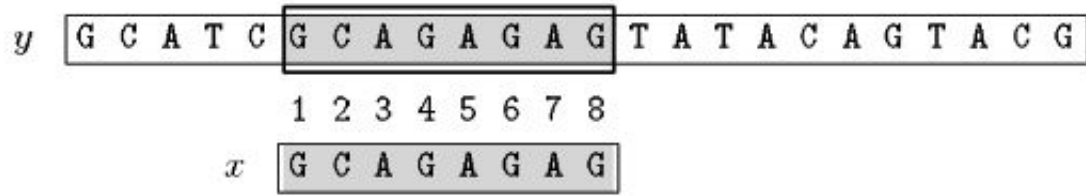
Hình 2.3: Lần 3: Dịch 1 bước



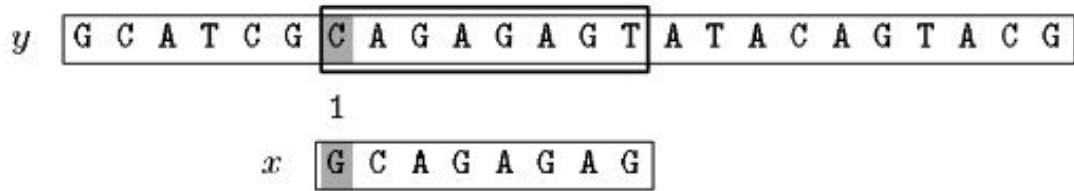
Hình 2.4: Lần 4: Dịch 1 bước



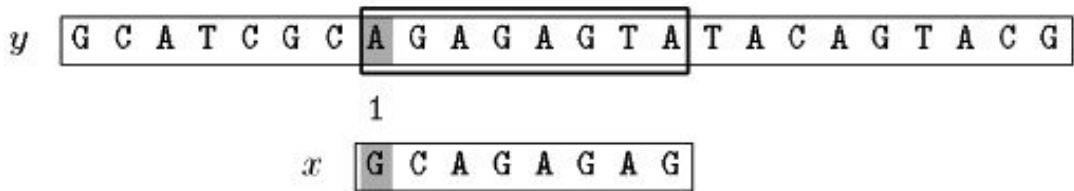
Hình 2.5: Lần 5: Dịch 1 bước



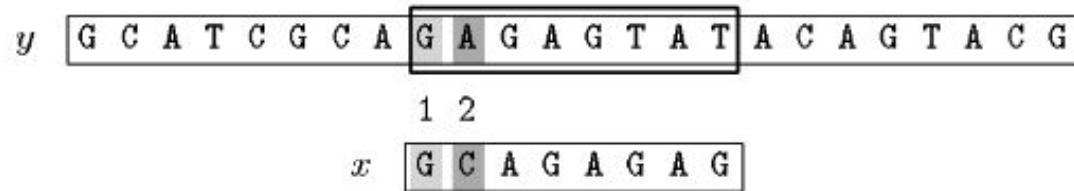
Hình 2.6: Lần 6: OUTPUT(5)



Hình 2.7: Lần 7: Dịch 1 bước



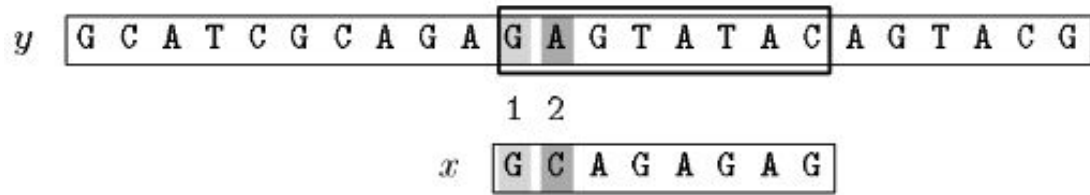
Hình 2.8: Lần 8: Dịch 1 bước



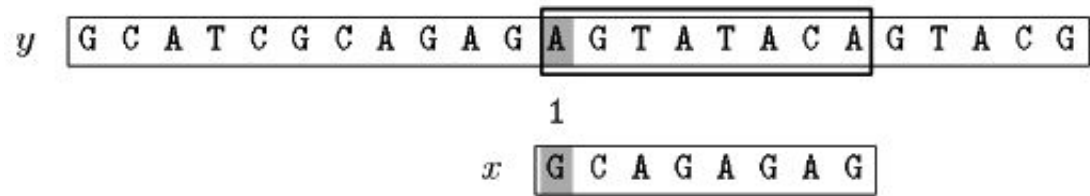
Hình 2.9: Lần 9: Dịch 1 bước



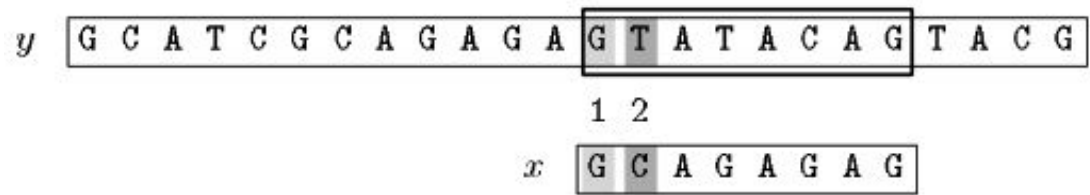
Hình 2.10: Lần 10: Dịch 1 bước



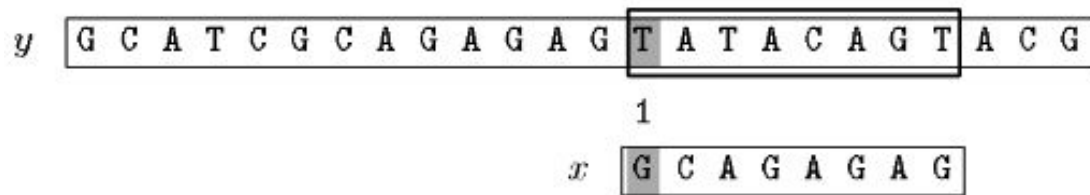
Hình 2.11: Lần 11: Dịch 1 bước



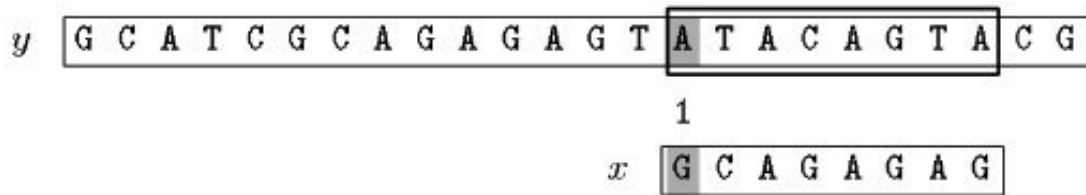
Hình 2.12: Lần 12: Dịch 1 bước



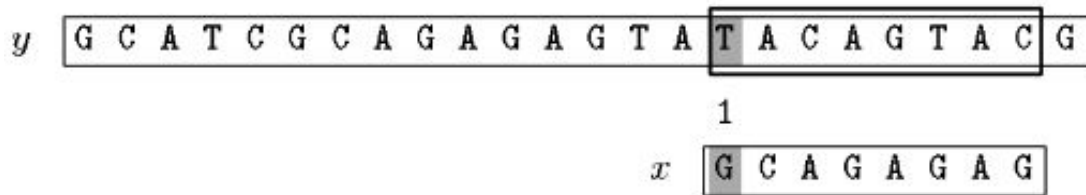
Hình 2.13: Lần 13: Dịch 1 bước



Hình 2.14: Lần 14: Dịch 1 bước



Hình 2.15: Lần 15: Dịch 1 bước



Hình 2.16: Lần 16: Dịch 1 bước



Hình 2.17: Lần 17: Kết thúc

Vậy ta thấy chỉ có lần 6 là tìm thấy mảng x trong mảng y . Trong ví dụ trên, thuật toán Brute Force phải thực hiện 30 lần so sánh.

2.1.4 Lập trình theo thuật toán

```

1 def BF(x, y):
2     m, n = len(x), len(y)
3     for j in range(n - m + 1):
4         if y[j:j + m] == x:
5             print(j)

```

2.2 Search with an automaton

2.2.1 Trình bày thuật toán

Thuật toán này xây dựng DFA (Deterministic Finite Automaton – máy trạng thái hữu hạn xác định) giúp tìm kiếm mẫu trên văn bản một cách nhanh chóng.

Định nghĩa máy trạng thái hữu hạn DFA:

DFA là một bộ gồm $(Q, \Sigma, \delta : Q \times \Sigma \rightarrow Q, q_0 \in Q, F \subseteq Q)$. Trong đó:

- Q – tập tất cả các tiền tố của mẫu $x = x_0 \dots x_{m-1}$: $Q = \{\epsilon, x_0, x_0x_1, \dots, x_0x_1 \dots x_{m-1}\}$
- $q_0 = \epsilon$ – trạng thái biểu diễn tiền tố rỗng.
- $F = x$ – trạng thái biểu diễn tiền tố trùng với mẫu x .
- Σ là tập các ký tự có trong mẫu x và văn bản y .
- δ – hàm chuyển tiếp: với mỗi $q \in Q$ và $c \in \Sigma$ thì $\delta(q, c) = qc$ khi $qc \in Q$; ngược lại $\delta(q, c) = p$ là hậu tố dài nhất của qc mà cũng là tiền tố của x .

Trong thuật toán này, quá trình tìm kiếm được đưa về một quá trình biến đổi trạng thái automaton. Mỗi trạng thái của automaton sẽ đại diện cho số ký tự đang khớp của mẫu với văn bản. Khi đạt được trạng thái cuối cùng F có nghĩa là đã tìm được một vị trí xuất hiện của mẫu.

2.2.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý có độ phức tạp là $O(m \times \sigma)$.
- Pha tìm kiếm có độ phức tạp là $O(n)$ nếu DFA được chứa trong bảng truy cập trực tiếp, ngược lại $O(n \times \log \sigma)$.

2.2.3 Kiểm nghiệm thuật toán

Giả sử ta tìm kiếm mẫu $x = abaa$ trong văn bản $y = ababbaabaaab$.

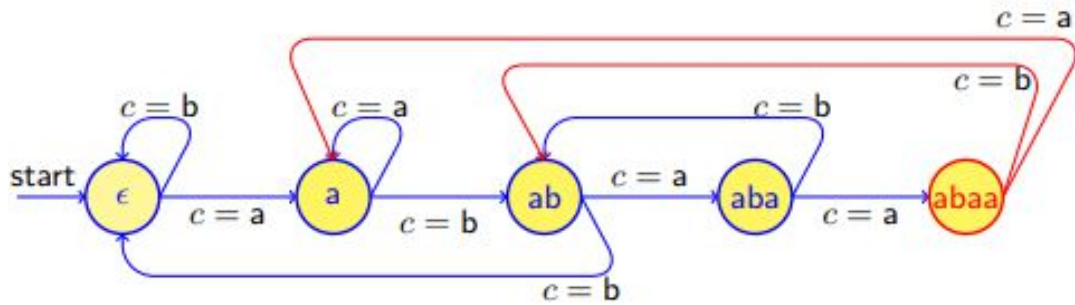
Pha tiền xử lý – tính DFA:

$$\Sigma = \{a, b\}; \quad Q = \{\epsilon, a, ab, aba, abaa\}; \quad q_0 = \epsilon; \quad F = abaa$$

Hàm chuyển tiếp $\delta(q, c)$:

$$q_{\text{next}} = \delta(q_{\text{curr}}, c) =$$

$q_{\text{curr}} \setminus c$	a	b
ϵ	a	ϵ
a	a	ab
ab	aba	ϵ
aba	abaa	ab
abaa	a	ab



Hình 2.18: Sơ đồ DFA cho mẫu $x = abaa$

Pha tìm kiếm:

BƯỚC	VĂN BẢN	TRẠNG THÁI
1	<u>a</u> babbaabaaaab	$\epsilon \rightarrow a$
2	<u>ab</u> abbaabaaaab	$a \rightarrow ab$
3	<u>aba</u> bbaabaaaab	$ab \rightarrow aba$
4	ab <u>ab</u> baabaaaab	$aba \rightarrow ab$
5	ababbaabaaaab	$ab \rightarrow \epsilon$
6	ababb <u>a</u> abaaaab	$\epsilon \rightarrow a$
7	ababba <u>a</u> baaab	$a \rightarrow a$
8	ababba <u>ab</u> aaaab	$a \rightarrow ab$
9	ababba <u>aba</u> aab	$ab \rightarrow aba$
10	ababba <u>abaa</u> ab	$aba \rightarrow \text{abaa}$
11	ababba <u>abaaa</u> b	$\text{abaa} \rightarrow a$
12	ababbaabaa <u>ab</u>	$a \rightarrow ab$

Tại bước thứ 11, thuật toán đã tìm thấy mẫu x trong văn bản y .

2.2.4 Lập trình theo thuật toán

```

1 def get_next_state(pat, m, state, c):
2     if state < m and c == pat[state]:
3         return state + 1
4     for ns in range(state, 0, -1):
5         if pat[ns - 1] == c and pat[:ns - 1] == pat[state - ns + 1:state]:
6             return ns
7     return 0
8
9 def compute_tf(pat):
10    m = len(pat)
11    tf = [[0] * 256 for _ in range(m + 1)]
12    for state in range(m + 1):
13        for c in range(256):
14            tf[state][c] = get_next_state(pat, m, state, chr(c))
15    return tf
16
17 def finite_automata(pat, txt):
18    m, n = len(pat), len(txt)
19    tf = compute_tf(pat)
20    state = 0
21    for i in range(n):
22        state = tf[state][ord(txt[i])]
23        if state >= m:
24            print(i - m + 1)

```

2.3 Thuật toán Karp-Rabin

2.3.1 Trình bày thuật toán

Trong thuật toán Brute Force, trường hợp xấu nhất là $O(m \times n)$. Thuật toán Karp-Rabin giải quyết vấn đề này bằng cách đưa cửa sổ và mẫu thành 1 giá trị băm để so sánh. Ngay khi thấy không khớp, nó sẽ chuyển sang cửa sổ mới.

Công thức tính giá trị băm:

$$\text{hash}(w[0..m-1]) = (w[0] \times 2^{m-1} + w[1] \times 2^{m-2} + \dots + w[m-1] \times 2^0) \bmod q$$

với q là một số nguyên tố lớn. Công thức cập nhật:

$$\text{rehash}(a, b, h) = ((h - a \times 2^{m-1}) \times 2 + b) \bmod q$$

trong đó w là cửa sổ có độ dài m , h là giá trị băm của cửa sổ trước đó, a là ký tự đầu tiên của cửa sổ trước đó, b là ký tự cuối cùng của cửa sổ tiếp theo.

2.3.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý có độ phức tạp $O(m)$.

- Pha tìm kiếm có độ phức tạp $O(m \times n)$, mong đợi $O(m + n)$ trong thực tế.

2.3.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$ ($m = 8$) và $y = \text{GCATCGCAGAGAGTATACAGTACG}$ ($n = 24$):

Giá trị băm của mẫu: $\text{hash}(\text{GCAGAGAG}) = 17597$.

Pha tìm kiếm:

Lần	Cửa sổ $y[i..i + 7]$	$\text{hash}(y[i..i + 7])$	Khớp?
1	GCATCGCA	17819	Không
2	CATCGCAG	17533	Không
3	ATCGCAGA	17979	Không
4	TCGCAGAG	19389	Không
5	CGCAGAGA	17339	Không
6	GCAGAGAG	17597	CÓ $\Rightarrow$ OUTPUT(5)
7	CAGAGAGT	17102	Không
8	AGAGAGTA	17117	Không
9	GAGAGTAT	17678	Không
10	AGAGTATA	17245	Không
11	GAGTATAC	17917	Không
12	AGTATACA	17723	Không
13	GTATACAG	18877	Không
14	TATACAGT	19662	Không
15	ATACAGTA	17885	Không
16	TACAGTAC	19197	Không
17	ACAGTACG	16961	Không

Thuật toán thực hiện 17 lần so sánh giá trị băm và 8 lần so sánh ký tự.

2.3.4 Lập trình theo thuật toán

```

1 def KR(x, y):
2     m, n = len(x), len(y)
3     d = 1 << (m - 1)
4     hx = hy = 0
5     for i in range(m):
6         hx = (hx << 1) + ord(x[i])
7         hy = (hy << 1) + ord(y[i])
8     j = 0
9     while j <= n - m:
10        if hx == hy and x == y[j:j + m]:

```

```

11     print(j)
12     if j + m < n:
13         hy = ((hy - ord(y[j]) * d) << 1) + ord(y[j + m])
14         j += 1

```

2.4 Thuật toán Shift Or

2.4.1 Trình bày thuật toán

Thuật toán Shift Or sử dụng kỹ thuật bitwise. Sử dụng 2 loại bit véc tơ là S và R , cả hai đều có độ dài m :

- Véc tơ S_c : $S_c[i] = 0$ nếu $x[i] = c$, ngược lại $S_c[i] = 1$. Véc tơ S_c chỉ ra vị trí của ký tự c trong mẫu x .
- Véc tơ R_j biểu diễn trạng thái sau khi đọc ký tự thứ j của văn bản:

$$R_j[i] = \begin{cases} 0 & \text{nếu } x[0..i] = y[j - i..j] \\ 1 & \text{ngược lại} \end{cases}$$

- Công thức cập nhật: $R_{j+1} = \text{Shift}(R_j) \text{ OR } S_{y[j+1]}$
- Nếu $R_j[m - 1] = 0$ thì tìm thấy mẫu tại vị trí $j - m + 1$.

2.4.2 Đánh giá độ phức tạp thuật toán

- Độ phức tạp pha tiền xử lý là $O(m + \sigma)$.
- Độ phức tạp pha tìm kiếm là $O(n)$.

2.4.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng các véc tơ S_c :

$x[i]$	S_A	S_C	S_G	S_T
G	1	1	0	1
C	1	0	1	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1

Pha tìm kiếm:

Bảng các véc tơ R_j :

		j																							
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
		G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
0	G	0	1	1	1	1	0	1	1	0	1	0	1	0	1	1	1	1	1	1	0	1	1	1	0
1	C	1	0	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	A	1	1	0	1	1	1	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1
3	G	1	1	1	1	1	0	1	1	0	1	0	1	0	1	1	1	1	1	1	0	1	1	1	0
4	A	1	1	1	1	1	1	1	1	1	0	1	0	1	0	1	1	1	1	1	1	1	1	1	1
5	G	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1
6	A	1	1	1	1	1	1	1	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1
7	G	1	1	1	1	1	1	1	1	1	1	1	1	①	1	1	1	1	1	1	1	1	1	1	1

Ta thấy $R_{12}[7] = 0$ có nghĩa rằng tìm thấy x ở vị trí $12 - 8 + 1 = 5$.

2.4.4 Lập trình theo thuật toán

```

1 def S0(x, y):
2     m, n = len(x), len(y)
3     MASK = (1 << 32) - 1
4     S = {}
5     lim = j_bit = 0
6     j_bit = 1
7     for i in range(m):
8         c = x[i]
9         S.setdefault(c, MASK)
10        S[c] = (S[c] & ~j_bit) & MASK
11        lim = (lim | j_bit) & MASK
12        j_bit = (j_bit << 1) & MASK
13    lim = ~(lim >> 1) & MASK

```

```

14 state = MASK
15 for j in range(n):
16     c = y[j]
17     state = ((state << 1) | S.get(c, MASK)) & MASK
18     if state < lim:
19         print(j - m + 1)

```

2.5 Thuật toán Morris-Pratt

2.5.1 Trình bày thuật toán

Thuật toán Morris-Pratt cải thiện số lần dịch chuyển của sổ bằng cách ghi nhớ một số phần của văn bản mà đã khớp với mẫu.

Giả sử tại bước so sánh, ta có $x[0..i-1] = y[j..i+j-1] = u$ và bắt đầu không khớp tại $a = x[i] \neq y[i+j] = b$.

Định nghĩa border: Với chuỗi $x[0..i-1]$, border là phần prefix dài nhất của x mà match với suffix của $x[0..i-1]$.

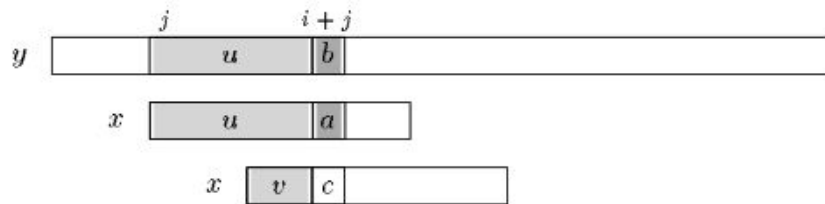


Figure 6.1 Shift in the Morris-Pratt algorithm: v is the border of u .

Hình 2.19: Dịch chuyển trong thuật toán Morris-Pratt: v là border của u

Gọi $\text{mpNext}[i]$ là độ dài của border của $x[0..i-1]$ ($0 < i \leq m$). Thuật toán thực hiện tối đa $2n - 1$ phép so sánh ký tự.

2.5.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(m + n)$.

2.5.3 Kiểm nghiệm thuật toán

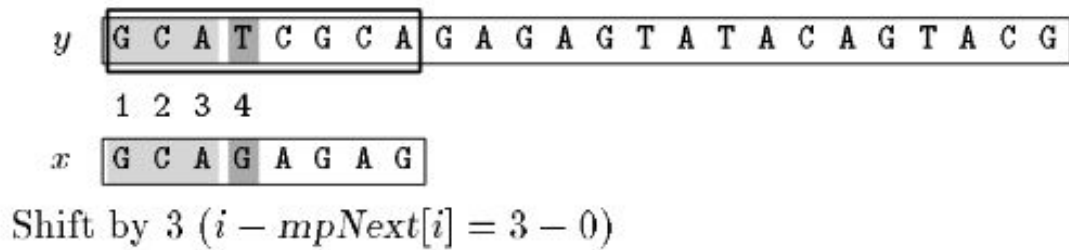
Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

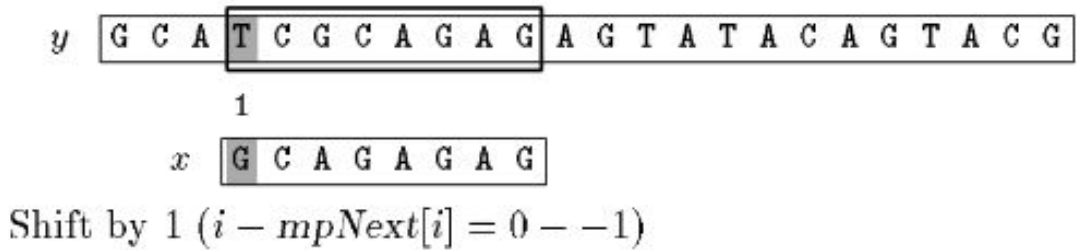
Bảng $\text{mpNext}[i]$ – độ dài border của $x[0..i-1]$:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$\text{mpNext}[i]$	-1	0	0	0	1	0	1	0	1

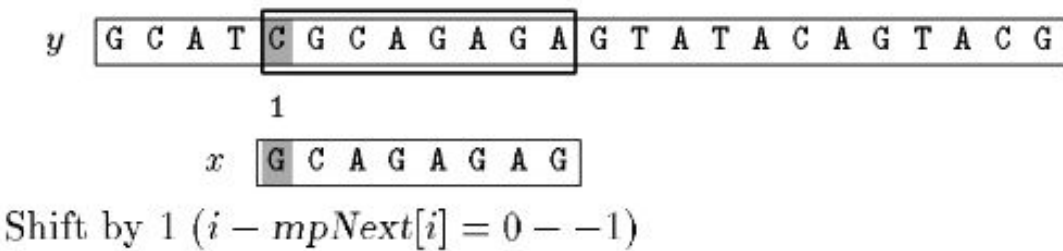
Pha tìm kiếm:



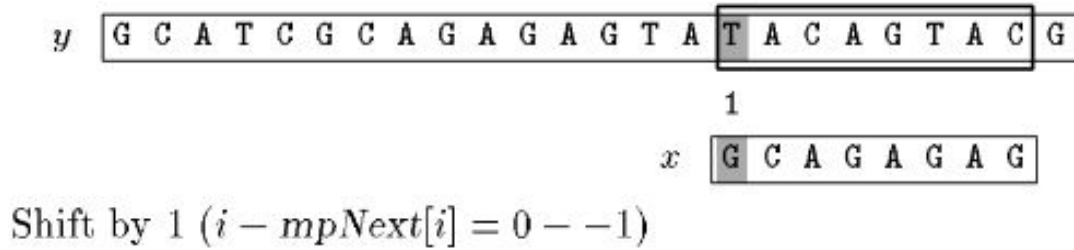
Hình 2.20: Lần 1: Mismatch tại $i=3$, dịch $i - \text{mpNext}[i] = 3 - 0 = 3$ bước



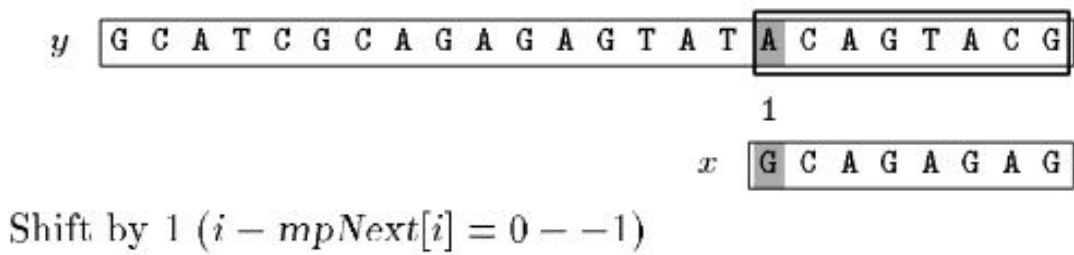
Hình 2.21: Lần 2: Dịch $0 - (-1) = 1$ bước



Hình 2.22: Lần 3: Dịch 1 bước



Hình 2.27: Lần 8: Dịch 1 bước



Hình 2.28: Lần 9: Dịch 1 bước

Thuật toán Morris-Pratt thực hiện 19 lần so sánh ký tự trong ví dụ trên.

2.5.4 Lập trình theo thuật toán

```

1 def pre_mp(x):
2     m = len(x)
3     mp_next = [-1] + [0] * m
4     i, j = 0, -1
5     while i < m:
6         while j > -1 and x[i] != x[j]:
7             j = mp_next[j]
8         i += 1
9         j += 1
10        mp_next[i] = j
11    return mp_next
12
13 def MP(x, y):
14     m, n = len(x), len(y)
15     mp_next = pre_mp(x)
16     i = j = 0
17     while j < n:
18         while i > -1 and x[i] != y[j]:
19             i = mp_next[i]
20         i += 1

```

```

21     j += 1
22     if i >= m:
23         print(j - i)
24         i = mp_next[i]

```

2.6 Thuật toán Knuth-Morris-Pratt

2.6.1 Trình bày thuật toán

Thuật toán này là một sự cải tiến của thuật toán Morris-Pratt.

Giả sử ta đang đặt cửa sổ tại vị trí j của văn bản y và $x[0..i-1] = y[j..i+j-1] = u$ và $a = x[i] \neq y[i+j] = b$.

Định nghĩa tagged border: Tagged border là phần prefix của mẫu x mà match với phần suffix của u (giống với định nghĩa border của thuật toán Morris-Pratt).

Ta gọi $\text{kmpNext}[i]$ là độ dài của tagged border của $x[0..i-1]$ theo sau bởi ký tự c khác ký tự $x[i]$. Ngược lại thì $\text{kmpNext}[i] = -1$ ($0 < i \leq m$). Khi đó ta có thể tránh việc so sánh c với $y[i+j]$ (vì nếu $c = x[i]$ thì đương nhiên $c \neq y[i+j]$ nên ta không cần so sánh nữa).

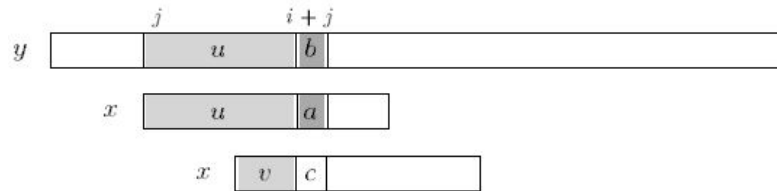


Figure 7.1 Shift in the Knuth-Morris-Pratt algorithm: v is a border of u and $a \neq c$.

Hình 2.29: Dịch chuyển trong thuật toán Knuth-Morris-Pratt: v là border của u và $a \neq c$

2.6.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(m + n)$.

2.6.3 Kiểm nghiệm thuật toán

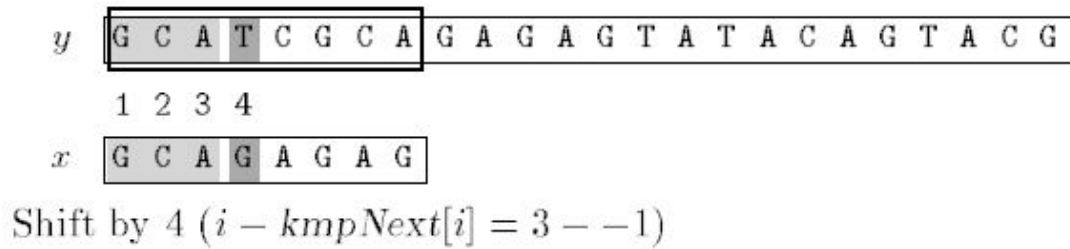
Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

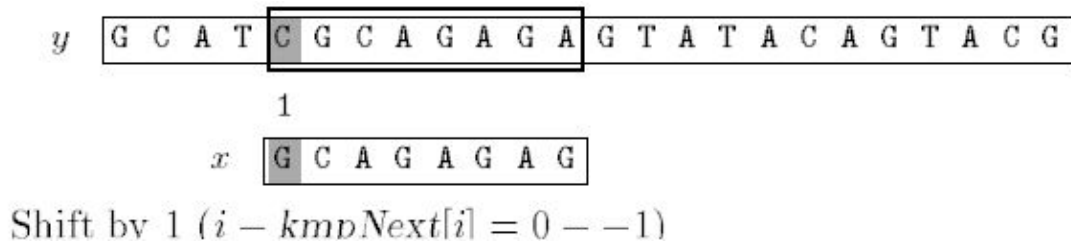
Bảng $kmpNext[i]$ – tagged border của $x[0..i-1]$:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$kmpNext[i]$	-1	0	0	-1	1	-1	1	-1	1

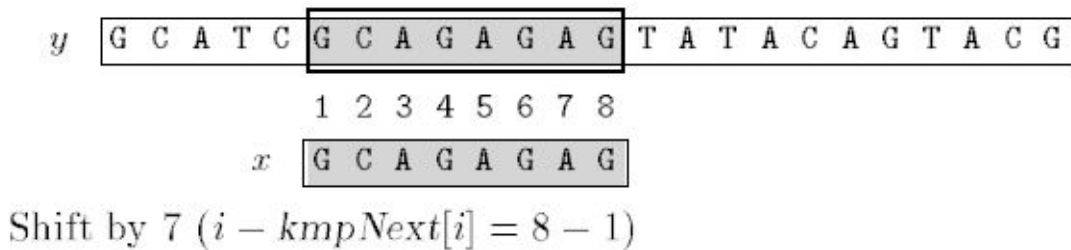
Pha tìm kiếm:



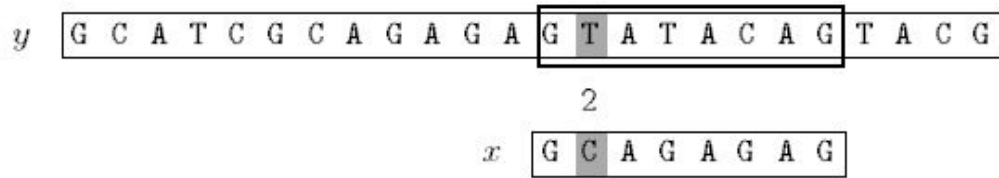
Hình 2.30: Lần 1: Dịch $3 - (-1) = 4$ bước



Hình 2.31: Lần 2: Dịch 1 bước

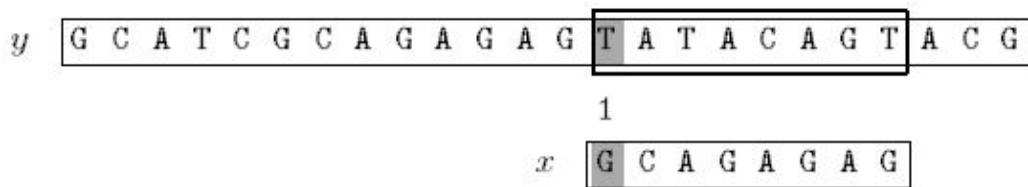


Hình 2.32: Lần 3: OUTPUT(5), dịch 7 bước



Shift by 1 ($i - kmpNext[i] = 1 - 0$)

Hình 2.33: Lần 4: Dịch 1 bước



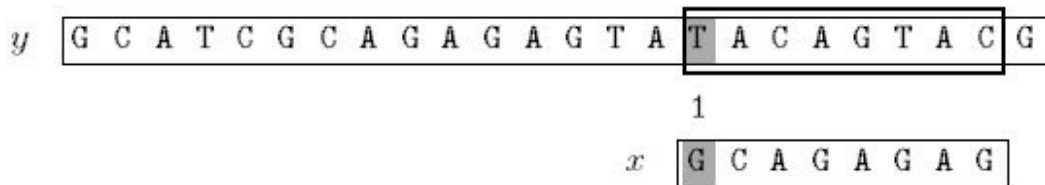
Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Hình 2.34: Lần 5: Dịch 1 bước



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

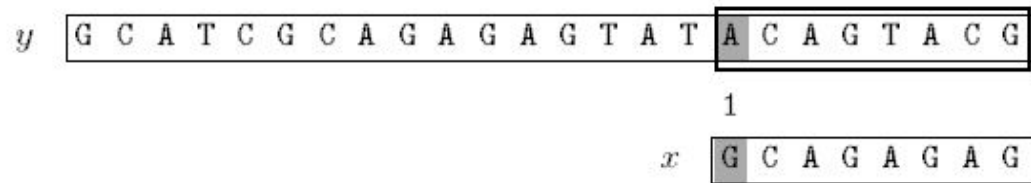
Hình 2.35: Lần 6: Dịch 1 bước



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Hình 2.36: Lần 7: Dịch 1 bước

Eighth attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Hình 2.37: Lần 8: Dịch 1 bước

Thuật toán KMP thực hiện 18 lần so sánh ký tự trong ví dụ trên.

2.6.4 Lập trình theo thuật toán

```

1 def pre_kmp(x):
2     m = len(x)
3     kmp_next = [-1] + [0] * m
4     i, j = 0, -1
5     while i < m:
6         while j > -1 and x[i] != x[j]:
7             j = kmp_next[j]
8         i += 1
9         j += 1
10        if i < m and x[i] == x[j]:
11            kmp_next[i] = kmp_next[j]
12        else:
13            kmp_next[i] = j
14    return kmp_next
15
16 def KMP(x, y):
17     m, n = len(x), len(y)
18     kmp_next = pre_kmp(x)
19     i = j = 0
20     while j < n:
21         while i > -1 and x[i] != y[j]:
22             i = kmp_next[i]
23         i += 1
24         j += 1
25         if i >= m:
26             print(j - i)
27             i = kmp_next[i]

```

2.7 Thuật toán Apostolico-Crochemore

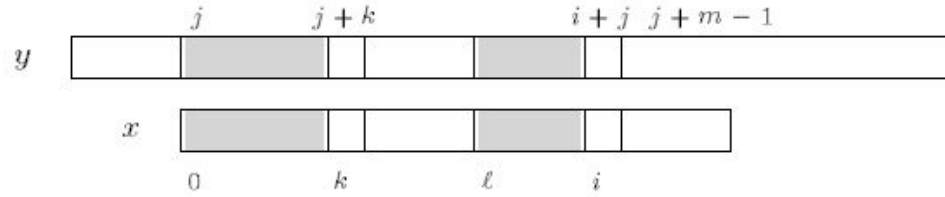
2.7.1 Trình bày thuật toán

Thuật toán Apostolico-Crochemore sử dụng bảng kmpNext trong thuật toán KMP để tính độ dịch chuyển của cửa sổ.

Đặt $\ell = 0$ nếu x là lũy thừa của 1 ký tự ($x = c^m$). Ngược lại, đặt $\ell =$ vị trí của ký tự đầu tiên trong x mà khác $x[0]$. Mỗi lần thử, thuật toán so sánh theo thứ tự: $\ell, \ell + 1, \dots, m - 1, 0, 1, \dots, \ell - 1$.

Trong pha tìm kiếm, thuật toán duy trì bộ ba (i, j, k) , trong đó:

- Cửa sổ được đặt ở đoạn $y[j..j + m - 1]$;
- $0 \leq k \leq \ell$ và $x[0..k - 1] = y[j..j + k - 1]$;
- $\ell \leq i < m$ và $x[\ell..i - 1] = y[j + \ell..i + j - 1]$.



Hình 2.38: Minh họa bộ ba (i, j, k) : vùng xám biểu diễn các đoạn đã khớp trong y và x

Ban đầu bộ ba được khởi tạo $(i, j, k) = (\ell, 0, 0)$. Cách tính bộ ba tiếp theo dựa vào bộ ba trước đó:

- **Trường hợp $i = \ell$:**
 - Nếu $x[i] = y[i + j]$ thì bộ ba tiếp theo là $(i + 1, j, k)$.
 - Nếu $x[i] \neq y[i + j]$ thì bộ ba tiếp theo là $(\ell, j + 1, \max\{0, k - 1\})$.
- **Trường hợp $\ell < i < m$:**
 - Nếu $x[i] = y[i + j]$ thì bộ ba tiếp theo là $(i + 1, j, k)$.
 - Nếu $x[i] \neq y[i + j]$, xét $\text{kmpNext}[i]$:
 - * Nếu $\text{kmpNext}[i] \leq \ell$: bộ ba tiếp theo là

$$(\ell, i + j - \text{kmpNext}[i], \max\{0, \text{kmpNext}[i]\}).$$

* Nếu $\text{kmpNext}[i] > \ell$: bộ ba tiếp theo là

$$(\text{kmpNext}[i], i + j - \text{kmpNext}[i], \ell).$$

• Trường hợp $i = m$:

- Nếu $k < \ell$ và $x[k] = y[j + k]$ thì bộ ba tiếp theo là $(i, j, k + 1)$.
- Ngược lại (hoặc $k = \ell$): nếu $k = \ell$ thì thông báo tìm thấy mẫu tại vị trí j . Trong cả hai trường hợp, bộ ba tiếp theo được tính như trong trường hợp $\ell < i < m$.

2.7.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(n)$.

2.7.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

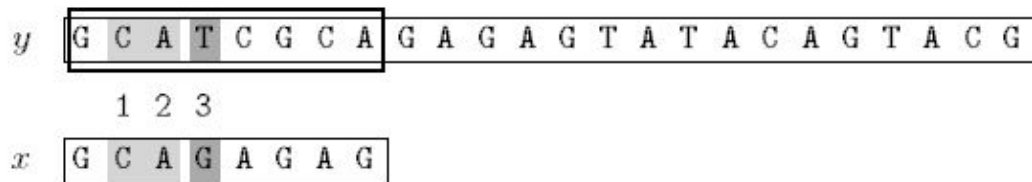
Pha tiền xử lý:

Bảng $\text{kmpNext}[i]$ với $\ell = 1$:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$\text{kmpNext}[i]$	-1	0	0	-1	1	-1	1	-1	1

Pha tìm kiếm:

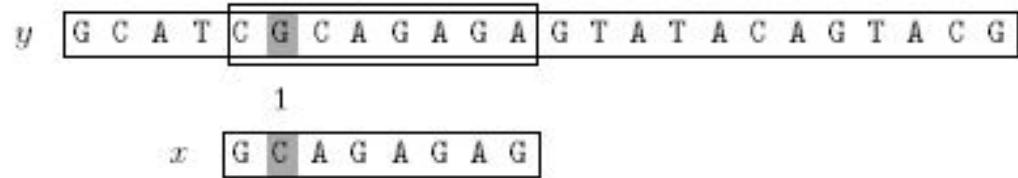
First attempt:



Shift by 4 ($i - \text{kmpNext}[i] = 3 - (-1)$)

Hình 2.39: Lần 1: Dịch 4 bước

Second attempt:



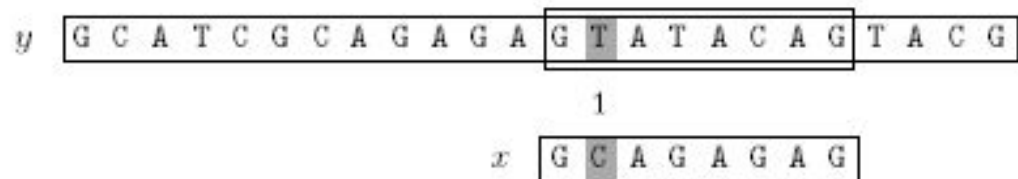
Shift by 1 ($i - kmpNext[i] = 1 - 0$)

Third attempt:



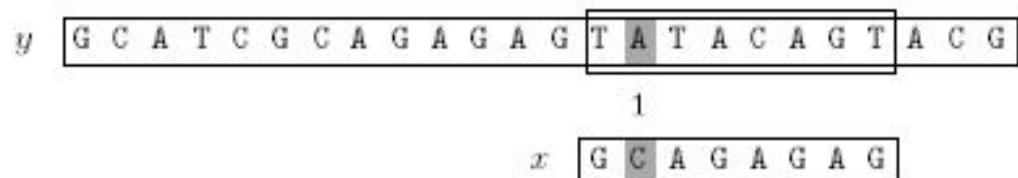
Shift by 7 ($i - kmpNext[i] = 8 - 1$)

Fourth attempt:



Shift by 1 ($i - kmpNext[i] = 1 - 0$)

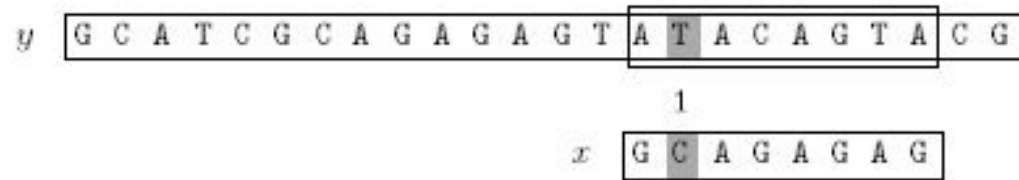
Fifth attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Hình 2.40: Lần 2: Dịch 1 bước; lần 3: OUTPUT(5), dịch 7 bước; lần 4–5: Dịch 1 bước

Sixth attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Seventh attempt:



Shift by 1 ($i - kmpNext[i] = 0 - -1$)

Hình 2.41: Lần 6–7: Dịch 1 bước

Eighth attempt:



Shift by 3 ($i - kmpNext[i] = 4 - 1$)

Hình 2.42: Lần 8: Dịch 3 bước – Kết thúc

Thuật toán Apostolico-Crochemore thực hiện 20 phép so sánh ký tự trong ví dụ trên.

2.7.4 Lập trình theo thuật toán

```

1 def pre_kmp(x):
2     m = len(x)
3     kmp_next = [-1] * m
4     i, j = 0, -1

```

```

5     while i < m:
6         while j > -1 and x[i] != x[j]:
7             j = kmp_next[j]
8         i += 1
9         j += 1
10        if i < m and x[i] == x[j]:
11            kmp_next[i] = kmp_next[j]
12        else:
13            kmp_next[i] = j
14    return kmp_next
15
16 def AC(x, y):
17     m, n = len(x), len(y)
18     kmp_next = pre_kmp(x)
19     ell = 1
20     while ell < m and x[ell - 1] == x[ell]:
21         ell += 1
22     if ell == m:
23         ell = 0
24     i = ell
25     j = k = 0
26     while j <= n - m:
27         while i < m and x[i] == y[i + j]:
28             i += 1
29         if i >= m:
30             while k < ell and x[k] == y[j + k]:
31                 k += 1
32             if k >= ell:
33                 print(j)
34         if i == ell or kmp_next[i] <= ell:
35             j += i - kmp_next[i]
36             i = ell
37             k = k - 1 if k > 0 else 0
38         else:
39             j += i - kmp_next[i]
40             k = ell if kmp_next[i] > ell else kmp_next[i]
41             i = kmp_next[i]

```

2.8 Thuật toán Not So Naïve

2.8.1 Trình bày thuật toán

Trong pha tìm kiếm của thuật toán Not So Naïve, việc so sánh các ký tự của mẫu x được thực hiện theo thứ tự: $1, 2, \dots, m-2, m-1, 0$.

Mỗi lần thử (tức mỗi lần đặt cửa sổ vào đoạn $y[j..j+m-1]$ của văn bản y):

- Nếu $x[0] = x[1]$ và $x[1] \neq y[j+1]$ (suy ra $x[0] \neq y[j+1]$), hoặc nếu $x[0] \neq x[1]$ và $x[1] = y[j+1]$ (suy ra $x[0] \neq y[j+1]$): thì cửa sổ được dịch **2 vị trí**.
- Ngược lại: cửa sổ được dịch **1 vị trí**.

Ta thấy thuật toán Not So Naïve gần giống với thuật toán Brute Force nhưng có cải tiến một chút (vì vậy mới có tên là Not So Naïve).

2.8.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: độ phức tạp hằng số.
- Pha tìm kiếm: $O(m \times n)$.

2.8.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Từ mẫu x ta tính được $k = 1$, $\ell = 2$.

Pha tìm kiếm:

First attempt:

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 2 3

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2

Second attempt:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

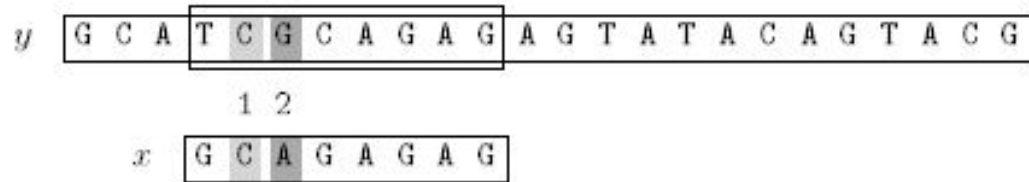
x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1

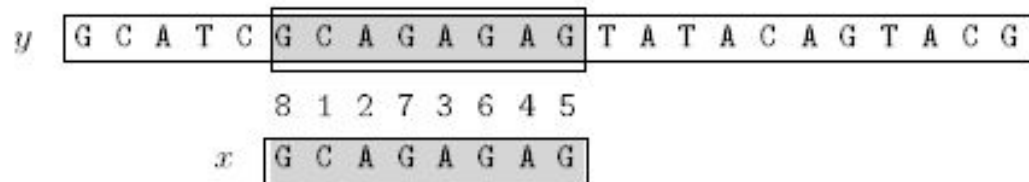
Hình 2.43: Lần 1: Dịch 2 bước; lần 2: Dịch 1 bước

Third attempt:



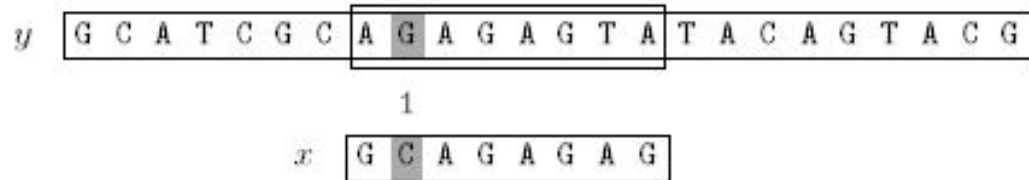
Shift by 2

Fourth attempt:



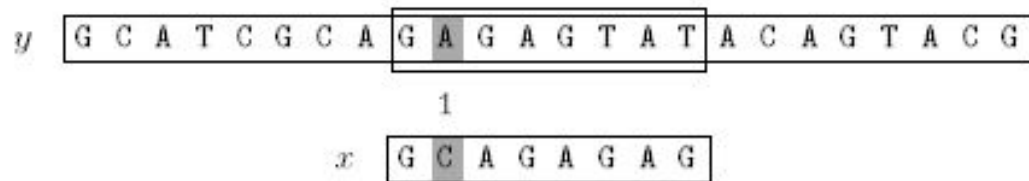
Shift by 2

Fifth attempt:



Shift by 1

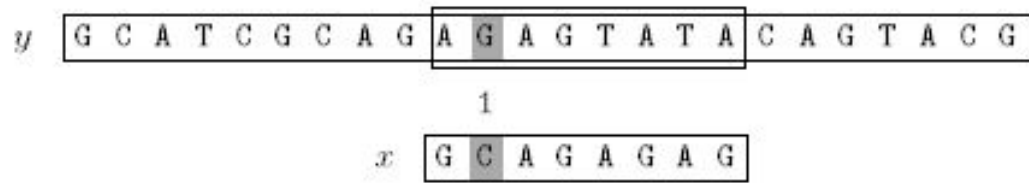
Sixth attempt:



Shift by 1

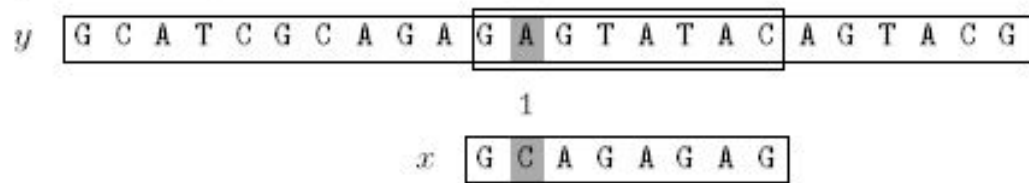
Hình 2.44: Lần 3-4: Dịch 2 bước; lần 5-6: Dịch 1 bước

Seventh attempt:



Shift by 1

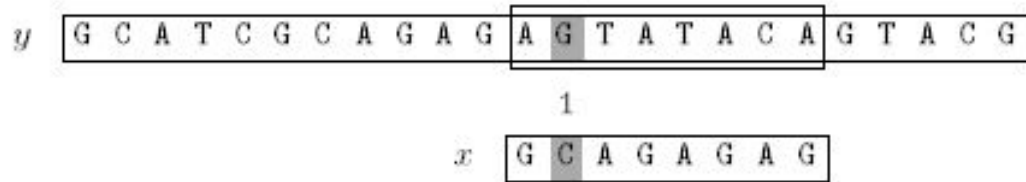
Eighth attempt:



Shift by 1

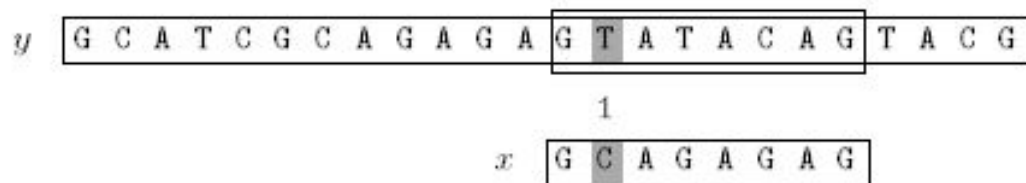
Hình 2.45: Lần 7–8: Dịch 1 bước

Ninth attempt:



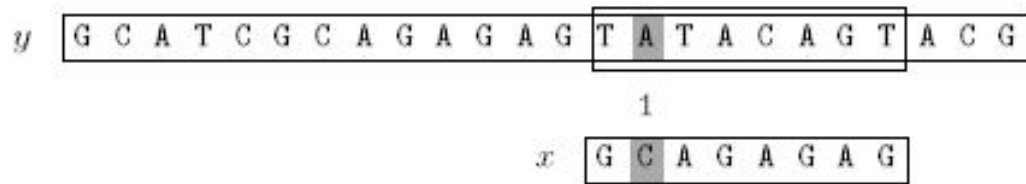
Shift by 1

Tenth attempt:



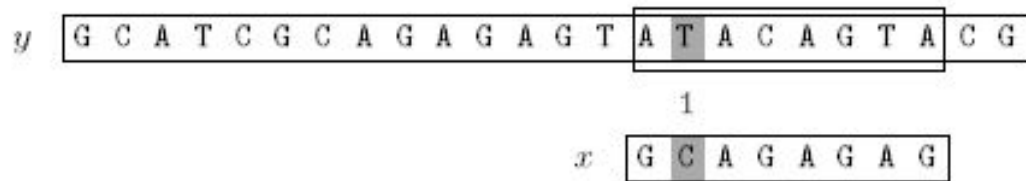
Shift by 1

Eleventh attempt:



Shift by 1

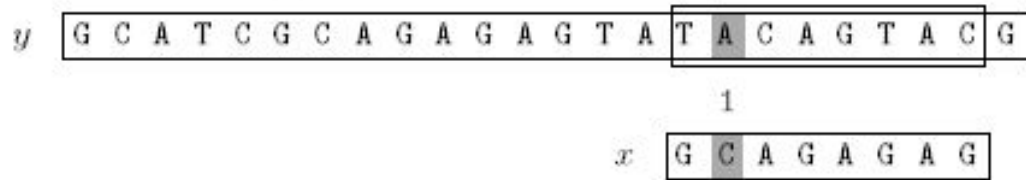
Twelfth attempt:



Shift by 1

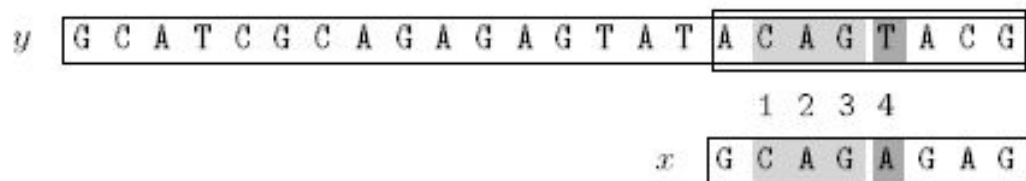
Hình 2.46: Lần 9–12: Dịch 1 bước

Thirteenth attempt:



Shift by 1

Fourteenth attempt:



Shift by 2

Hình 2.47: Lần 13: Dịch 1 bước; lần 14: Dịch 2 bước – Kết thúc

Ta thấy thuật toán Not So Naïve thực hiện 27 phép so sánh ký tự trong ví dụ trên.

2.8.4 Lập trình theo thuật toán

```

1 def NSN(x, y):
2     m, n = len(x), len(y)
3     if x[0] == x[1]:
4         k, ell = 2, 1
5     else:
6         k, ell = 1, 2
7     j = 0
8     while j <= n - m:
9         if x[1] != y[j + 1]:
10            j += k
11        else:
12            if x[2:] == y[j + 2:j + m] and x[0] == y[j]:
13                print(j)
14            j += ell

```

Chương 3

Tìm kiếm mẫu từ phải sang trái

3.1 Thuật toán Boyer-Moore

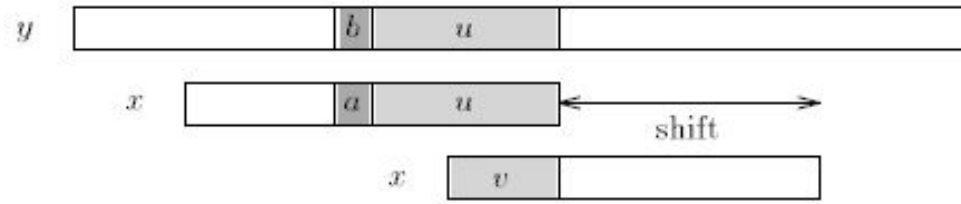
3.1.1 Trình bày thuật toán

Thuật toán Boyer-Moore so sánh các ký tự của pattern x với văn bản y từ phải sang trái và dựa trên 2 kiểu dịch chuyển cửa sổ là good-suffix shift và bad-character shift.

Good-suffix shift: Giả sử sự không khớp xảy ra ở cặp $a = x[i]$ và $y[i + j] = b$ trong lần thử ở vị trí j của văn bản y . Do đó $x[i + 1..m - 1] = y[i + j + 1..j + m - 1] = u$ và $x[i] \neq y[i + j]$. Good-suffix shift dùng thẳng đoạn $y[i + j + 1..j + m - 1] = x[i + 1..m - 1] = u$ với 1 segment bên phải nhất của x sao cho segment đó $= u$ và ký tự ngay trước nó khác $x[i]$. Nếu không có một segment nào như vậy thì sẽ dịch cửa sổ sao cho dùng thẳng suffix dài nhất v của đoạn $y[i + j + 1..j + m - 1]$ với một prefix của x mà match với suffix đó.

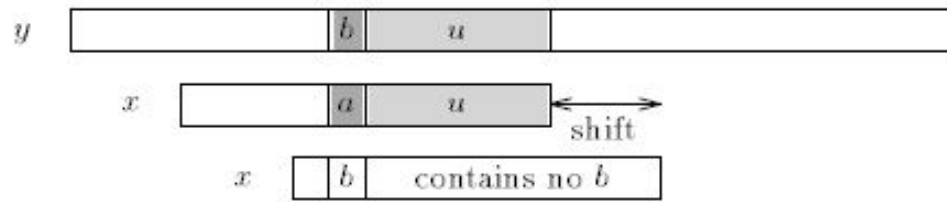


Hình 3.1: Good-suffix shift trong trường hợp tìm được 1 segment trong x mà ký tự c ngay trước nó khác a

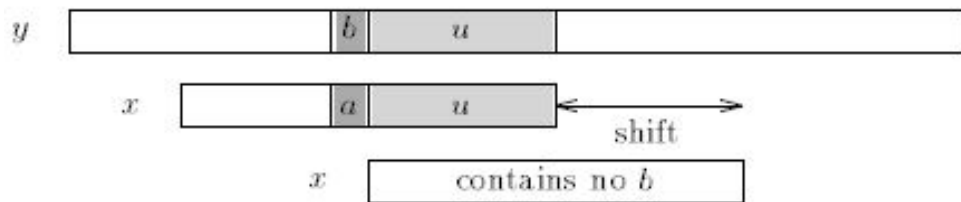


Hình 3.2: Good-suffix shift trong trường hợp không tìm được segment trong x thỏa mãn yêu cầu

Bad-character shift: Dùng ký tự $y[i + j] = b$ của văn bản y với 1 ký tự c bên phải nhất của đoạn $x[0..m - 2]$ sao cho $c = b$. Nếu không tìm thấy ký tự $b = y[i + j]$ trong mẫu x thì ta dịch cửa sổ ra ngay sau ký tự b tức cửa sổ bắt đầu ở ký tự $y[i + j + 1]$.



Hình 3.3: Bad-character shift trong trường hợp tìm thấy một ký tự b trong x thỏa mãn yêu cầu



Hình 3.4: Bad-character shift trong trường hợp không tìm thấy ký tự b trong x

Với mỗi lần thử ta có 2 cách dịch chuyển cửa sổ là good-suffix shift và bad-character shift. Đối với bad-character shift, cửa sổ có thể bị dịch lại (dịch sang trái). Ta sẽ chọn cách dịch chuyển mà dịch cửa sổ sang phải nhiều nhất.

3.1.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m + \sigma)$.
- Pha tìm kiếm: $O(m \times n)$ trường hợp xấu nhất, $O(n/m)$ trường hợp trung bình.

3.1.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng bad-character shift $\text{bmBc}[a]$ – khoảng cách từ vị trí xuất hiện phải nhất của a trong $x[0..m-2]$ đến cuối mẫu:

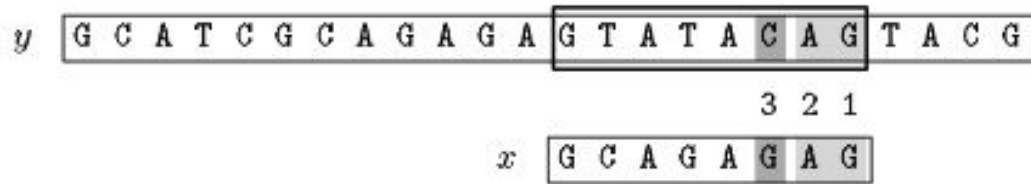
a	A	C	G	T
bmBc	1	6	2	8

Bảng good-suffix shift $\text{bmGs}[i]$:

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$\text{suff}[i]$	1	0	0	2	0	4	0	8
$\text{bmGs}[i]$	7	7	7	2	7	4	7	1

Pha tìm kiếm:

Fourth attempt:



Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Fifth attempt:



Shift by 7 ($bmGs[6]$)

Hình 3.6: Lần 4–5: Kết thúc

Ta thấy thuật toán Boyer-Moore thực hiện 17 phép so sánh ký tự trong ví dụ trên.

3.1.4 Lập trình theo thuật toán

```

1 def suffixes(x):
2     m = len(x)
3     suff = [0] * m
4     suff[m - 1] = m
5     g = m - 1
6     f = 0
7     for i in range(m - 2, -1, -1):
8         if i > g and suff[i + m - 1 - f] < i - g:
9             suff[i] = suff[i + m - 1 - f]
10        else:
11            if i < g:
12                g = i
13            f = i
14            while g >= 0 and x[g] == x[g + m - 1 - f]:
15                g -= 1
16            suff[i] = f - g
17    return suff
18
19 def pre_bm_gs(x):

```

```

20     m = len(x)
21     suff = suffixes(x)
22     bm_gs = [m] * m
23     j = 0
24     for i in range(m - 1, -1, -1):
25         if suff[i] == i + 1:
26             while j < m - 1 - i:
27                 if bm_gs[j] == m:
28                     bm_gs[j] = m - 1 - i
29                 j += 1
30     for i in range(m - 1):
31         bm_gs[m - 1 - suff[i]] = m - 1 - i
32     return bm_gs
33
34 def pre_bm_bc(x):
35     m = len(x)
36     bm_bc = {}
37     for i in range(m):
38         bm_bc[x[i]] = m - 1 - i
39     return bm_bc
40
41 def BM(x, y):
42     m, n = len(x), len(y)
43     bm_gs = pre_bm_gs(x)
44     bm_bc = pre_bm_bc(x)
45     j = 0
46     while j <= n - m:
47         i = m - 1
48         while i >= 0 and x[i] == y[i + j]:
49             i -= 1
50         if i < 0:
51             print(j)
52             j += bm_gs[0]
53         else:
54             j += max(bm_gs[i],
55                     bm_bc.get(y[i + j], m) - m + 1 + i)

```

3.2 Thuật toán Turbo-BM

3.2.1 Trình bày thuật toán

Thuật toán Turbo-BM là sự cải tiến của thuật toán Boyer-Moore. Thuật toán này cốt yếu ở việc nó ghi nhớ đoạn text của y (chính là đoạn u trong thuật toán Boyer-Moore) mà match với suffix của x ở lần thử trước đó (chỉ khi một good-suffix shift được thực hiện). Kỹ thuật này có 2 lợi thế:

- Nó có thể nhảy qua đoạn text đó (đoạn u).
- Nó cho phép thực hiện một turbo-shift.

Định nghĩa turbo-shift: Turbo-shift chỉ có thể xuất hiện nếu ở lần thử hiện tại suffix của pattern mà match với text là ngắn hơn suffix ở lần thử trước đó. Trong trường hợp này chúng ta gọi u là suffix trước đó và v là suffix ở lần thử hiện tại. Gọi a và b là các ký tự gây ra sự không khớp (mismatch) trong lần thử hiện tại (a thuộc x và b thuộc y). Thì av là suffix của x và av cũng là suffix của u (do độ dài v bé hơn độ dài u). Hai ký tự a và b cách nhau khoảng cách p trong text, và suffix của x có độ dài $|uzv|$ có 1 đoạn độ dài $p = |zv|$. Vì u là border của $uzv \Rightarrow$ sẽ không tìm thấy x trong y khi ký tự b chạy sang trái 1 đoạn $|u| - |v| - 1$. Vậy độ dịch nhỏ nhất có thể có độ dài là $|u| - |v|$, ta gọi nó là turbo-shift.

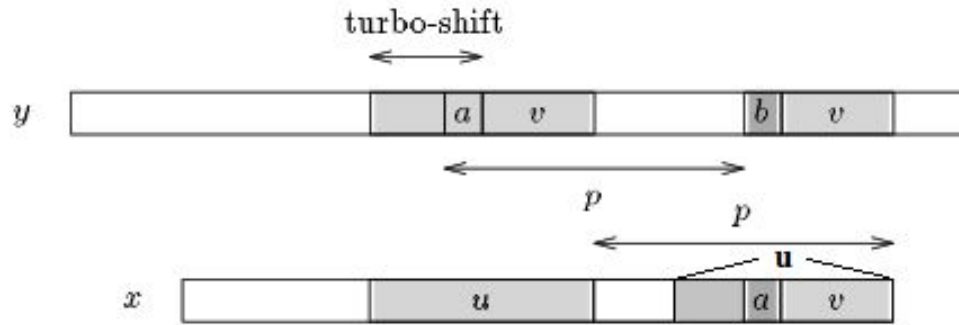


Figure 15.1 A turbo-shift can apply when $|v| < |u|$.

Hình 3.7: Turbo-shift khi $|v| < |u|$

Xét trường hợp $|v| < |u|$, nếu độ dài của bad-character shift lớn hơn độ dài của good-suffix shift và turbo-shift thì độ dài dịch chuyển của số thực sự phải là $|u| + 1$. Thật vậy, trong trường hợp này 2 ký tự c và d là khác nhau và ta đang giả sử rằng lần dịch trước là good-suffix shift. Thì sau đó một sự dịch chuyển dài hơn turbo-shift nhưng ngắn hơn $|u| + 1$ sẽ dùng c và d vào vị trí với cùng một ký tự trong v , do vậy sẽ không tìm thấy một sự match nào. Do đó, trường hợp này độ dài dịch chuyển thực sự ít nhất phải bằng $|u| + 1$.

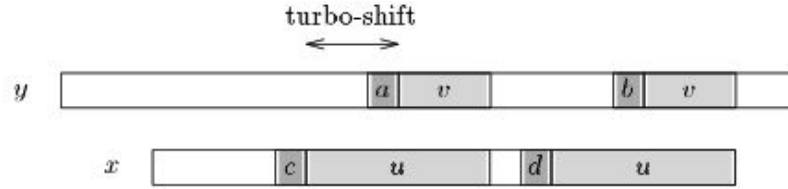


Figure 15.2 $c \neq d$ so they cannot be aligned with the same character in v .

Hình 3.8: Turbo-shift: $c \neq d$ không thể căn chỉnh với cùng ký tự trong v

Như vậy ta thấy rằng nếu $|v| \geq |u|$, ta so sánh good-suffix shift và bad-character shift để chọn ra cái tốt nhất. Ngược lại nếu $|v| < |u|$, ta so sánh turbo-shift, good-suffix shift, bad-character shift và $|u| + 1$ để chọn ra cái tốt nhất.

3.2.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m + \sigma)$.
- Pha tìm kiếm: $O(n)$.

3.2.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

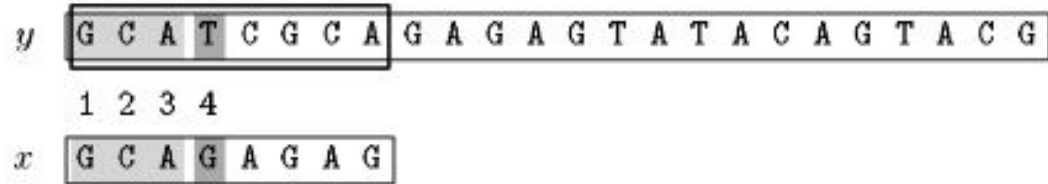
Pha tiền xử lý:

Bảng quick search bad-character shift $\text{qsBc}[a]$ – khoảng cách từ vị trí xuất hiện phải nhất của a trong toàn bộ x đến cuối mẫu cộng 1:

a	A	C	G	T
$\text{qsBc}[a]$	2	7	1	9

Pha tìm kiếm:

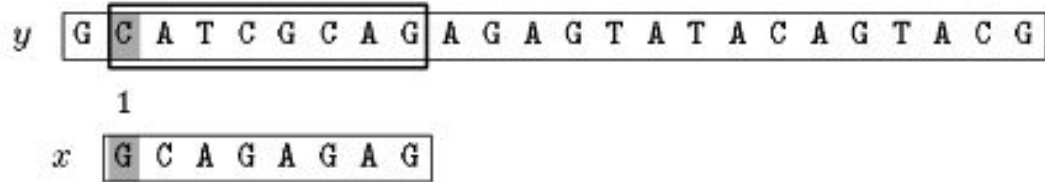
First attempt:



Shift by 1 ($qsBc[G]$)

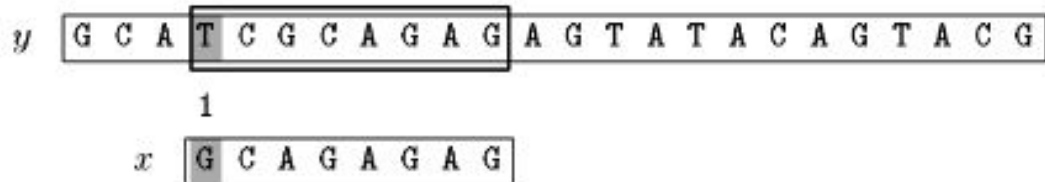
Hình 3.9: Lần 1: Dịch 3 bước

Second attempt:



Shift by 2 ($qsBc[A]$)

Third attempt:



Shift by 2 ($qsBc[A]$)

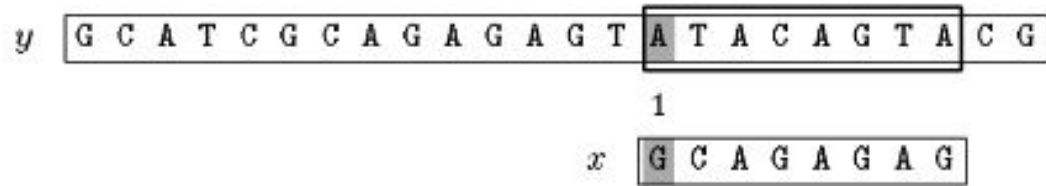
Fourth attempt:



Shift by 9 ($qsBc[T]$)

Hình 3.10: Lần 2: Dịch 2 bước; lần 3: Dịch 2 bước; lần 4: OUTPUT(5), dịch 9 bước

Fifth attempt:



Hình 3.11: Lần 5: Dịch 7 bước – Kết thúc

Ta thấy thuật toán Turbo-Boyer-Moore thực hiện 15 phép so sánh ký tự trong ví dụ trên.

3.2.4 Lập trình theo thuật toán

```

1 def pre_bm_gs(x):
2     m = len(x)
3     suff = suffixes(x)
4     bm_gs = [m] * m
5     j = 0
6     for i in range(m - 1, -1, -1):
7         if suff[i] == i + 1:
8             while j < m - 1 - i:
9                 if bm_gs[j] == m:
10                     bm_gs[j] = m - 1 - i
11                 j += 1
12     for i in range(m - 1):
13         bm_gs[m - 1 - suff[i]] = m - 1 - i
14     return bm_gs
15
16 def pre_bm_bc(x):
17     m = len(x)
18     bm_bc = {}
19     for i in range(m):
20         bm_bc[x[i]] = m - 1 - i
21     return bm_bc
22
23 def TBM(x, y):
24     m, n = len(x), len(y)
25     bm_gs = pre_bm_gs(x)
26     bm_bc = pre_bm_bc(x)
27     j = u = 0
28     shift = m
29     while j <= n - m:
30         i = m - 1
31         while i >= 0 and x[i] == y[i + j]:

```

```

32     i -= 1
33     if u != 0 and i == m - 1 - shift:
34         i -= u
35     if i < 0:
36         print(j)
37         shift = bm_gs[0]
38         u = m - shift
39     else:
40         v = m - 1 - i
41         turbo_shift = u - v
42         bc_shift = bm_bc.get(y[i + j], m) - m + 1 + i
43         shift = max(turbo_shift, bc_shift)
44         shift = max(shift, bm_gs[i])
45         if shift == bm_gs[i]:
46             u = min(m - shift, v)
47         else:
48             if turbo_shift < bc_shift:
49                 shift = max(shift, u + 1)
50             u = 0
51     j += shift

```

3.3 Thuật toán Quick Search

3.3.1 Trình bày thuật toán

Là phiên bản đơn giản hóa của thuật toán Boyer-Moore, chỉ sử dụng bad-character shift, dễ dàng cài đặt.

Ta thấy rằng sau mỗi lần thử (tức cửa sổ được đặt ở đoạn $y[j..j+m-1]$ của text), độ dài dịch chuyển ít nhất phải bằng 1, vậy nên ký tự $y[j+m]$ đương nhiên sẽ được xét trong lần thử tiếp theo. Và vì vậy $y[j+m]$ có thể được sử dụng cho bad-character shift của lần thử hiện tại. Bad-character shift của thuật toán này được sửa đổi một chút như sau:

$$\text{qsBc}[c] = \begin{cases} \min\{m-i : 0 \leq i < m-1, x[i] = c\} + 1 & \text{nếu } c \in x \\ m+1 & \text{nếu } c \notin x \end{cases}$$

Như vậy trong mỗi lần thử tại đoạn $y[j..j+m-1]$, nếu không match với pattern x ta tìm bad-character shift với bad-character shift = $\text{qsBc}[y[j+m]]$.

3.3.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: độ phức tạp thời gian $O(m + \sigma)$ và độ phức tạp bộ nhớ $O(\sigma)$.

- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

3.3.3 Kiểm nghiệm thuật toán

Pha tiền xử lý:

Bảng $qsBc[a]$ – quick search bad-character shift dựa vào ký tự liền sau của số $y[j + m]$:

a	A	C	G	T
$qsBc$	2	7	1	9

Pha tìm kiếm:

First attempt:



Shift by 1 ($qsBc[G]$)

Hình 3.12: Lần 1: Dịch 1 ($qsBc[G]$)

Ta thấy thuật toán Quick Search thực hiện 15 phép so sánh ký tự trong ví dụ trên.

3.3.4 Lập trình theo thuật toán

```

1 def pre_qs_bc(x):
2     m = len(x)
3     qs_bc = {}
4     for i in range(m):
5         qs_bc[x[i]] = m - i
6     return qs_bc
7
8 def QS(x, y):
9     m, n = len(x), len(y)
10    qs_bc = pre_qs_bc(x)
11    j = 0
12    while j <= n - m:
13        if x == y[j:j + m]:
14            print(j)
15        if j + m < n:
16            j += qs_bc.get(y[j + m], m + 1)
17        else:
18            break

```

3.4 Thuật toán Tuned Boyer-Moore

3.4.1 Trình bày thuật toán

Tuned-Boyer-Moore là phiên bản đơn giản hóa của thuật toán Boyer-Moore, dễ cài đặt và rất nhanh trong thực tế.

Phần tốn chi phí nhất trong các thuật toán đối sánh mẫu là kiểm tra xem liệu ký tự trong mẫu có khớp với ký tự trong cửa sổ không. Để tránh việc kiểm tra này nhiều lần, Tuned-Boyer-Moore sẽ dịch chuyển cửa sổ một số lần trước khi thực sự so sánh ký tự.

Thuật toán sử dụng bad-character shift để tìm ký tự $x[m-1]$ trong y . Thuật toán sẽ tiếp tục dịch chuyển cửa sổ cho đến khi tìm thấy $x[m-1]$ trong y . Thuật toán yêu cầu lưu giá trị của $\text{bmBc}[x[m-1]]$ vào biến `shift`, và sau đó đặt lại $\text{bmBc}[x[m-1]] = 0$. Khi $x[m-1]$ được tìm thấy trong y , $m-1$ ký tự còn lại được kiểm tra và sau đó dịch cửa sổ một đoạn `shift`.

3.4.2 Đánh giá độ phức tạp thuật toán

- Độ phức tạp thuật toán: $O(m \times n)$.

3.4.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

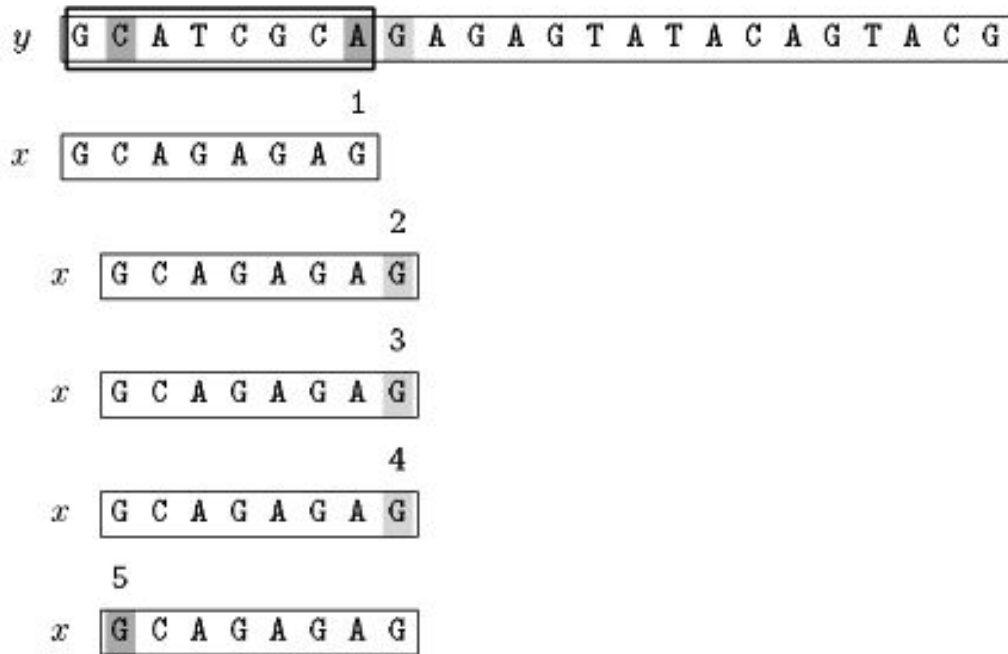
Bảng bad-character shift $\text{bmBc}[a]$: ký tự $x[m-1] = \text{G}$ được đặt bằng 0 thay vì giá trị thực, các ký tự khác tính như Horspool với $\text{shift} = 2$:

a	A	C	G	T
$\text{bmBc}[a]$	1	6	0	8

Pha tìm kiếm:

Searching phase

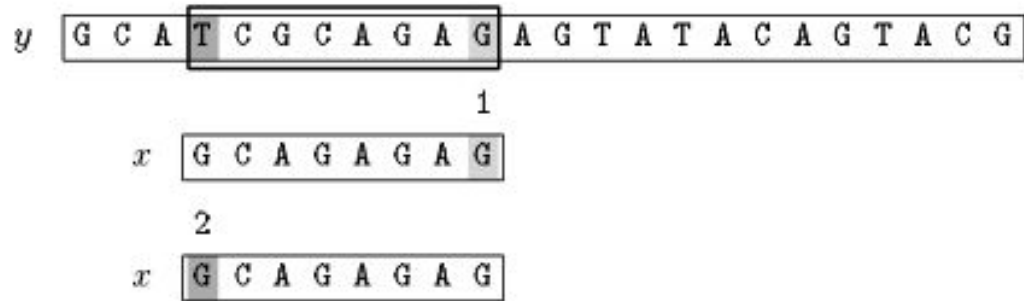
First attempt:



Shift by 2 (*shift*)

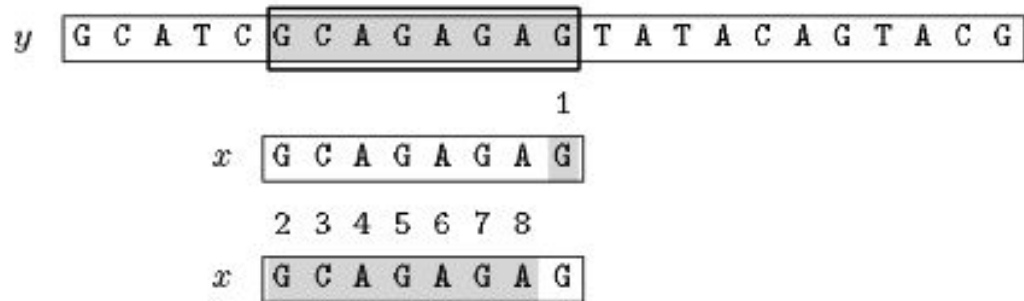
Hình 3.15: Lần 1: 5 phép so sánh, dịch 2 bước

Second attempt:



Shift by 2 (*shift*)

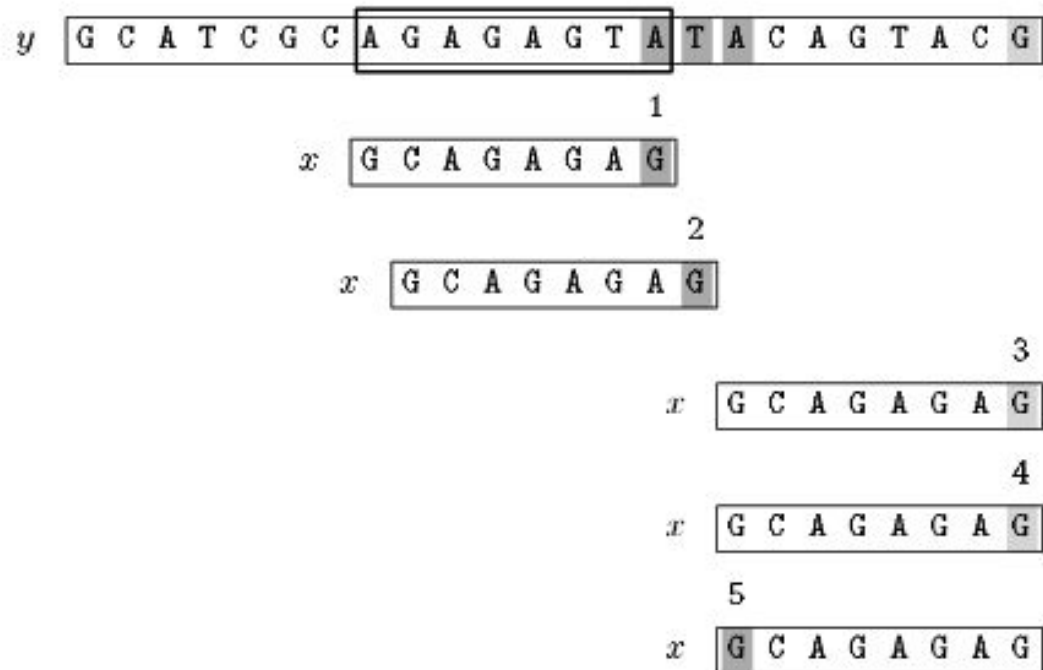
Third attempt:



Shift by 2 (*shift*)

Hình 3.16: Lần 2: Dịch 2 bước; lần 3: OUTPUT(5), dịch 2 bước

Fourth attempt:



Shift by 2 (*shift*)

Hình 3.17: Lần 4: Dịch 2 bước – Kết thúc

Ta thấy thuật toán Tuned-Boyer-Moore thực hiện 10 phép so sánh ký tự và 10 phép dịch chuyển trong ví dụ trên.

3.4.4 Lập trình theo thuật toán

```

1 def pre_bm_bc(x):
2     m = len(x)
3     bm_bc = {}
4     for i in range(m):
5         bm_bc[x[i]] = m - 1 - i
6     return bm_bc
7
8 def TUNEDBM(x, y):
9     m, n = len(x), len(y)
10    bm_bc = pre_bm_bc(x)
11    shift = bm_bc.get(x[m - 1], m)
12    bm_bc[x[m - 1]] = 0
13    sentinel = y + x[m - 1] * m
14    j = 0

```

```

15 while j <= n - m:
16     k = bm_bc.get(sentinel[j + m - 1], m)
17     while k != 0:
18         j += k; k = bm_bc.get(sentinel[j + m - 1], m)
19         j += k; k = bm_bc.get(sentinel[j + m - 1], m)
20         j += k; k = bm_bc.get(sentinel[j + m - 1], m)
21 if x[:m - 1] == y[j:j + m - 1] and j <= n - m:
22     print(j)
23 j += shift

```

3.5 Thuật toán Zhu-Takaoka

3.5.1 Trình bày thuật toán

Thuật toán Zhu-Takaoka là biến thể của thuật toán Boyer-Moore. Thuật toán sử dụng 2 ký tự liên tiếp nhau để tính bad-character shift.

Trong pha tìm kiếm, các so sánh được thực hiện từ phải sang trái trong cửa sổ $y[j..j+m-1]$, và sự không khớp xảy ra ở $x[m-k]$ và $y[j+m-k]$, và đoạn khớp là $x[m-k+1..m-1] = y[j+m-k+1..j+m-1]$. Thuật toán sẽ tính bad-character shift cho 2 ký tự cuối của cửa sổ là $y[j+m-2]$ và $y[j+m-1]$, đồng thời cũng tính được good-suffix shift. Độ dịch lớn hơn sẽ được chọn để dịch cửa sổ ($\max(\text{bad-character shift}, \text{good-suffix shift})$).

Công thức tính bad-character shift cho 2 ký tự $a, b \in \Sigma$:

$$\text{ztBc}[a, b] = k \Leftrightarrow \begin{cases} k < m - 2 & \text{và } x[m-k..m-k+1] = ab \\ & \text{và } ab \text{ không xuất hiện trong } x[m-k+2..m-2], \\ \text{hoặc} \\ k = m - 1 & x[0] = b \text{ và } ab \text{ không xuất hiện trong } x[0..m-2], \\ \text{hoặc} \\ k = m & x[0] \neq b \text{ và } ab \text{ không xuất hiện trong } x[0..m-2]. \end{cases}$$

3.5.2 Đánh giá độ phức tạp thuật toán

- Pha cài đặt: $O(m + \sigma^2)$.
- Pha thực thi: $O(m \times n)$.

3.5.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng two-character bad-character shift $\text{ztBc}[a][b]$ – shift dựa vào cặp ký tự $(y[j + m - 1], y[j + m])$ ngay sau và tại cuối của số:

ztBc	A	C	G	T
A	8	8	2	8
C	5	8	7	8
G	1	6	7	8
T	8	8	7	8

Bảng good-suffix shift $\text{bmGs}[i]$:

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$\text{bmGs}[i]$	7	7	7	2	7	4	7	1

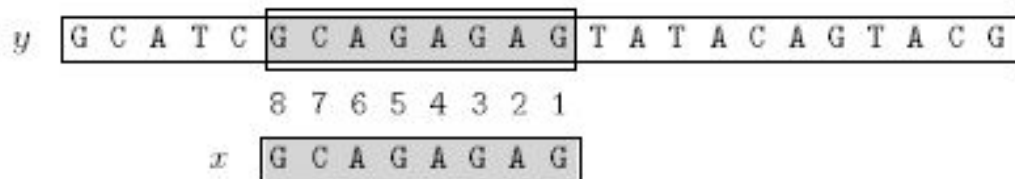
Pha tìm kiếm:

First attempt:



Shift by 5 ($\text{ztBc}[\text{C}][\text{A}]$)

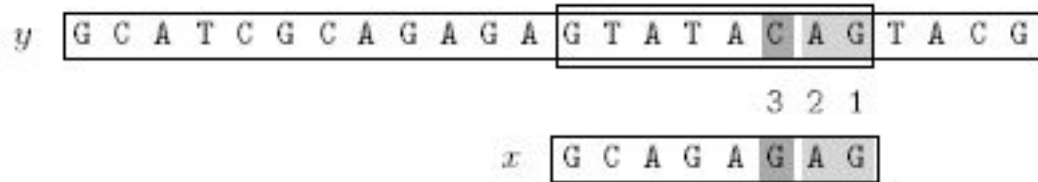
Second attempt:



Shift by 7 ($\text{bmGs}[0]$)

Hình 3.18: Lần 1: Dịch 5 bước ($\text{ztBc}[\text{C}][\text{A}]$); lần 2: $\text{OUTPUT}(5)$, dịch 7 bước

Third attempt:



Shift by 4 ($bmGs[6]$)

Fourth attempt:



Shift by 7 ($bmGs[7] = ztBc[C][G]$)

Hình 3.19: Lần 3–4: Kết thúc

Ta thấy thuật toán Zhu-Takaoka thực hiện 14 phép so sánh trong ví dụ trên.

3.5.4 Lập trình theo thuật toán

```

1 def pre_zt_bc(x):
2     m = len(x)
3     zt_bc = {}
4     for i in range(1, m - 1):
5         zt_bc[(x[i - 1], x[i])] = m - 1 - i
6     return zt_bc
7
8 def pre_bm_gs(x):
9     m = len(x)
10    suff = suffixes(x)
11    bm_gs = [m] * m
12    j = 0
13    for i in range(m - 1, -1, -1):
14        if suff[i] == i + 1:
15            while j < m - 1 - i:
16                if bm_gs[j] == m:
17                    bm_gs[j] = m - 1 - i
18                j += 1
19    for i in range(m - 1):

```

```

20     bm_gs[m - 1 - suff[i]] = m - 1 - i
21     return bm_gs
22
23 def zt_shift(zt_bc, a, b, x, m):
24     if (a, b) in zt_bc:
25         return zt_bc[(a, b)]
26     if b == x[0]:
27         return m - 1
28     return m
29
30 def ZT(x, y):
31     m, n = len(x), len(y)
32     zt_bc = pre_zt_bc(x)
33     bm_gs = pre_bm_gs(x)
34     j = 0
35     while j <= n - m:
36         i = m - 1
37         while i >= 0 and x[i] == y[i + j]:
38             i -= 1
39         if i < 0:
40             print(j)
41             j += bm_gs[0]
42         else:
43             j += max(bm_gs[i],
44                     zt_shift(zt_bc, y[j + m - 2], y[j + m - 1], x, m))

```

3.6 Thuật toán Berry-Ravindran

3.6.1 Trình bày thuật toán

Thuật toán Berry-Ravindran là thuật toán lai giữa 2 thuật toán Quick Search và Zhu-Takaoka. Thuật toán tính bad-character shift cho 2 ký tự liên tiếp nhau nằm ngay sau cửa sổ (là 2 ký tự $y[j + m]$ và $y[j + m + 1]$).

Pha tiền xử lý thực hiện tính độ dịch bad-character shift cho cặp ký tự (a, b) với $a, b \in \Sigma$. Cặp (a, b) chính là cặp $(y[j + m], y[j + m + 1])$. Ta đặt mẫu x vào giữa a, b (tức axb) và tìm sự xuất hiện bên phải nhất của ab trong $axb \Rightarrow$ từ đó tính được giá trị của bad-character shift cho (a, b) .

Cách tính bad-character shift cho (a, b) :

$$\text{brBc}[a, b] = \min \begin{cases} 1 & \text{nếu } x[m - 1] = a, \\ m - i & \text{nếu } x[i]x[i + 1] = ab \ (0 \leq i < m), \\ m + 1 & \text{nếu } x[0] = b, \\ m + 2 & \text{trong các trường hợp khác.} \end{cases}$$

Nếu cửa sổ hiện tại đang ở vị trí $y[j..j + m - 1]$ thì bước dịch cửa sổ tiếp theo sẽ có độ dài là $\text{brBc}[y[j + m], y[j + m + 1]]$.

3.6.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m + \sigma^2)$.
- Pha tìm kiếm: $O(m \times n)$.

3.6.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng $\text{brBc}[a][b]$ – shift dựa vào cặp ký tự $(y[j + m], y[j + m + 1])$ liền sau cửa sổ:

brBc	A	C	G	T	*
A	10	10	2	10	10
C	7	10	9	10	10
G	1	1	1	1	1
T	10	10	9	10	10
*	10	10	9	10	10

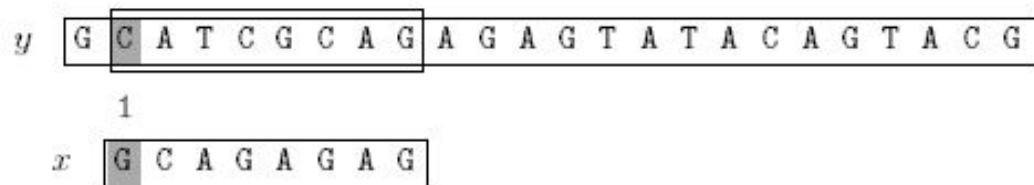
Pha tìm kiếm:

First attempt:



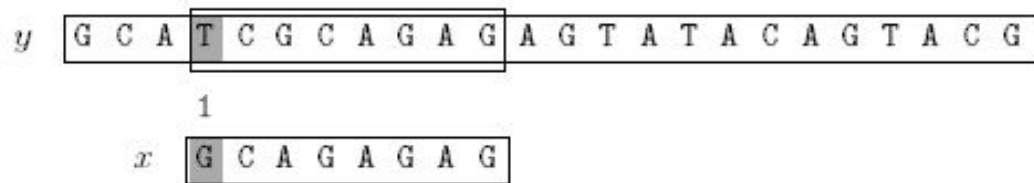
Shift by 1 ($brBc[G][A]$)

Second attempt:



Shift by 2 ($brBc[A][G]$)

Third attempt:



Shift by 2 ($brBc[A][G]$)

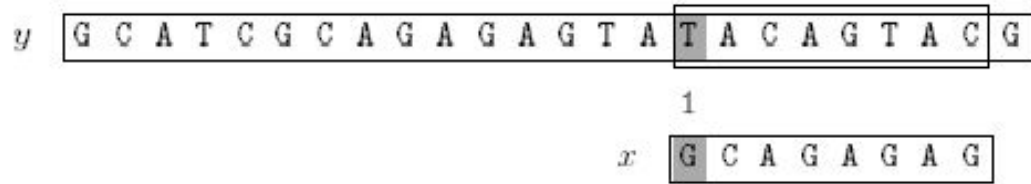
Fourth attempt:



Shift by 10 ($brBc[T][A]$)

Hình 3.20: Lần 1–4: OUTPUT(5) ở lần 4, dịch 10 bước

Fifth attempt:



Shift by 1 ($brBc[G][0]$)

Sixth attempt:



Shift by 10 ($brBc[0][0]$)

Hình 3.21: Lần 5–6: Kết thúc

Ta thấy thuật toán Berry-Ravindran thực hiện 16 phép so sánh ký tự trong ví dụ trên.

3.6.4 Lập trình theo thuật toán

```

1 def pre_br_bc(x):
2     m = len(x)
3     br_bc = {}
4     for i in range(m - 1):
5         br_bc[(x[i], x[i + 1])] = m - i
6     return br_bc
7
8 def br_shift(br_bc, a, b, x, m):
9     if a == x[m - 1]:
10        return 1
11    if (a, b) in br_bc:
12        return br_bc[(a, b)]
13    if b == x[0]:
14        return m + 1
15    return m + 2
16
17 def BR(x, y):
18     m, n = len(x), len(y)
19     br_bc = pre_br_bc(x)

```

```

20 y_ext = y + '\0\0'
21 j = 0
22 while j <= n - m:
23     if x == y[j:j + m]:
24         print(j)
25     j += br_shift(br_bc, y_ext[j + m], y_ext[j + m + 1], x, m)
    
```

3.7 Thuật toán Apostolico-Giancarlo

3.7.1 Trình bày thuật toán

Thuật toán Apostolico-Giancarlo là biến thể của thuật toán Boyer-Moore. Thuật toán Boyer-Moore khó phân tích và sau mỗi lần thử nó quên hết tất cả các ký tự đã khớp trước đó. Apostolico-Giancarlo ghi nhớ độ dài của suffix dài nhất của mẫu x sau mỗi lần thử. Thông tin này được lưu trong bảng `skip`.

Giả sử trong 1 lần thử ở vị trí nhỏ hơn j , thuật toán đã match một suffix độ dài k của x ở vị trí $i+j$ ($0 < i < m$) thì $\text{skip}[i+j] = k$. Nghĩa là $y[i+j-k+1..i+j] = x[m-1-k+1..m-1]$.

Đặt $\text{suff}[i]$ ($0 \leq i < m$) bằng độ dài suffix dài nhất của x kết thúc ở vị trí i trong x .

Trong lần thử tại vị trí j , nếu đoạn text $y[i+j+1..j+m-1]$ đã khớp với mẫu x thì 4 trường hợp sau có thể xảy ra (với $k = \text{skip}[i+j]$ và $s = \text{suff}[i]$):

- **TH 1:** $k > \text{suff}[i]$ và $\text{suff}[i] = i + 1$. Điều đó có nghĩa rằng ta đã tìm thấy một mẫu x trong y . Khi đó $\text{skip}[j+m-1] = m$.

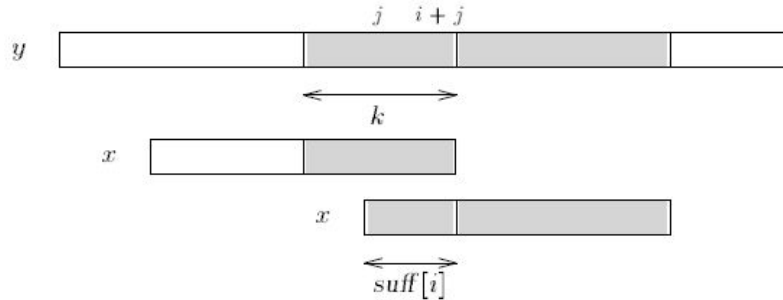


Figure 16.1 Case 1, $k > \text{suff}[i]$ and $\text{suff}[i] = i + 1$, an occurrence of x is found.

Hình 3.22: Trường hợp 1: $k > \text{suff}[i]$ và $\text{suff}[i] = i + 1$, tìm thấy một lần xuất hiện của x

- **TH 2:** $k > \text{suff}[i]$ và $\text{suff}[i] \leq i$. Điều đó có nghĩa là sự không khớp xảy ra giữa ký tự $x[i - \text{suff}[i]]$ và $y[i+j - \text{suff}[i]]$. Khi đó $\text{skip}[j+m-1] = m - 1 - i + \text{suff}[i]$. Bước dịch được thực hiện sử dụng $\text{bmBc}[y[i+j - \text{suff}[i]]]$ và $\text{bmGs}[i - \text{suff}[i] + 1]$.

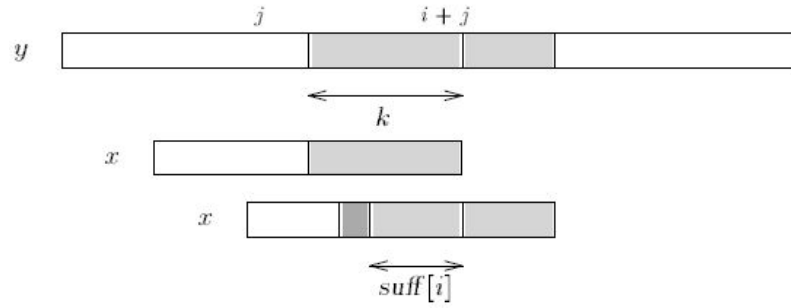


Figure 16.2 Case 2, $k > \text{suff}[i]$ and $\text{suff}[i] \leq i$, a mismatch occurs between $y[i + j - \text{suff}[i]]$ and $x[i - \text{suff}[i]]$.

Hình 3.23: Trường hợp 2: $k > \text{suff}[i]$ và $\text{suff}[i] \leq i$, mismatch xảy ra giữa $y[i + j - \text{suff}[i]]$ và $x[i - \text{suff}[i]]$

- **TH 3:** $k < \text{suff}[i]$, nghĩa là sự không khớp xảy ra giữa ký tự $x[i - k]$ và $y[i + j - k]$. Khi đó $\text{skip}[j + m - 1] = m - 1 - i + k$. Bước dịch được thực hiện sử dụng $\text{bmBc}[y[i + j - k]]$ và $\text{bmGs}[i - k + 1]$.

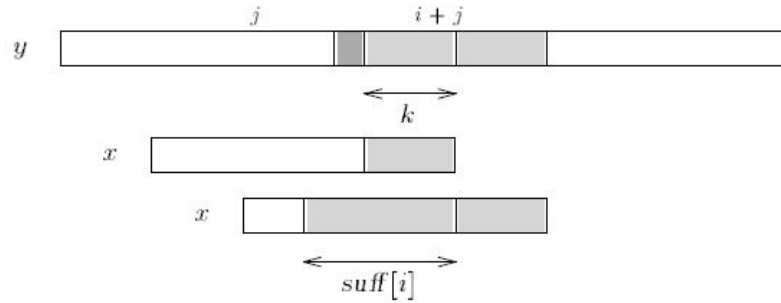


Figure 16.3 Case 3, $k < \text{suff}[i]$ a mismatch occurs between $y[i + j - k]$ and $x[i - k]$.

Hình 3.24: Trường hợp 3: $k < \text{suff}[i]$, mismatch xảy ra giữa $y[i + j - k]$ và $x[i - k]$

- **TH 4:** $k = \text{suff}[i]$. Đây là trường hợp duy nhất bước nhảy phải nhảy qua đoạn $y[i + j - k + 1..i + j]$ để so sánh 2 ký tự $y[i + j - k]$ và $x[i - k]$.

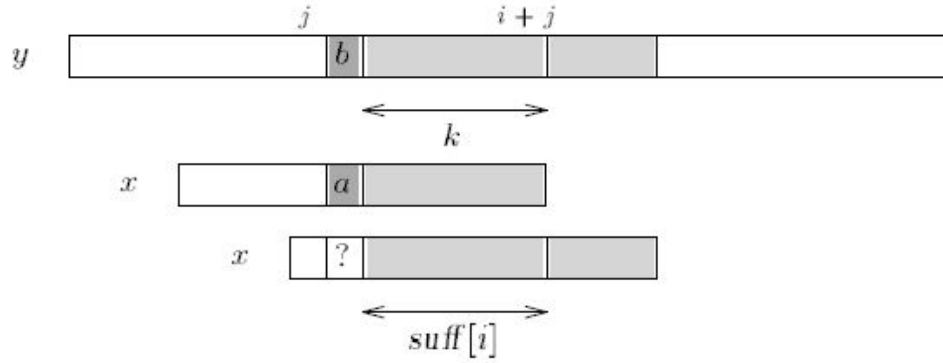


Figure 16.4 Case 4, $k = \text{suff}[i]$ and $a \neq b$.

Hình 3.25: Trường hợp 4: $k = \text{suff}[i]$ và $a \neq b$

Thuật toán Apostolico-Giancarlo sử dụng 2 cấu trúc dữ liệu:

- Một bảng **skip** được cập nhật ở cuối mỗi lần thử j theo công thức:

$$\text{skip}[j + m - 1] = \max \{ k : x[m - k .. m - 1] = y[j + m - k .. j + m - 1] \}$$

- Một bảng **suff** sử dụng để tính bảng **bmGs**, được định nghĩa với $1 \leq i < m$:

$$\text{suff}[i] = \max \{ k : x[i - k + 1 .. i] = x[m - k .. m - 1] \}$$

3.7.2 Đánh giá độ phức tạp thuật toán

- Pha xử lý: $O(m + \sigma)$.
- Pha tìm kiếm: $O(n)$, trường hợp xấu nhất $\frac{3}{2}n$ phép so sánh.

3.7.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng bad-character shift $\text{bmBc}[a]$:

a	A	C	G	T
$\text{bmBc}[a]$	1	6	2	8

Bảng good-suffix shift $bmGs[i]$:

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$suff[i]$	1	0	0	2	0	4	0	8
$bmGs[i]$	7	7	7	2	7	4	7	1

Pha tìm kiếm:

First attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G

1

x G C A G A G A G

Shift by 1 ($bmGs[7] = bmBc[A] - 7 + 7$)

Second attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G

3 2 1

x G C A G A G A G

Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Third attempt:

y G C A T C G C A G A G A G T A T A C A G T A C G

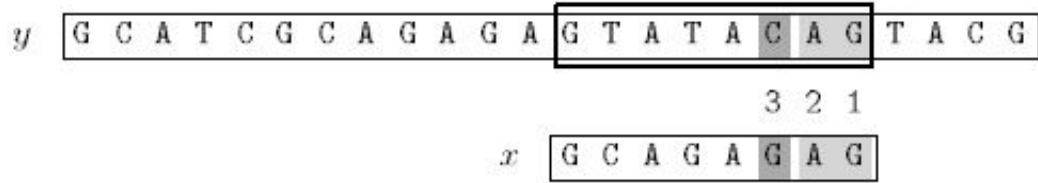
6 5 4 3 2 1

x G C A G A G A G

Shift by 7 ($bmGs[0]$)

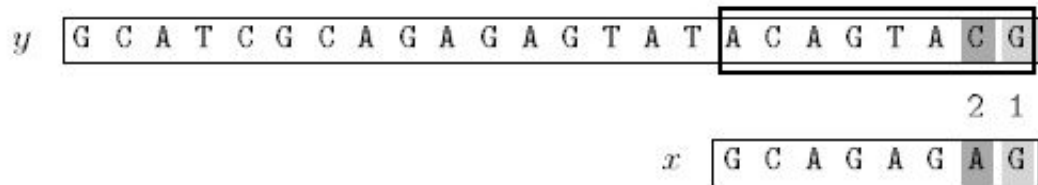
Hình 3.26: Lần 1-3: OUTPUT(5) ở lần 3

Fourth attempt:



Shift by 4 ($bmGs[5] = bmBc[C] - 7 + 5$)

Fifth attempt:



Shift by 7 ($bmGs[6]$)

Hình 3.27: Lần 4-5: Kết thúc

Ta thấy thuật toán Apostolico-Giancarlo thực hiện 15 phép so sánh trong ví dụ trên.

3.7.4 Lập trình theo thuật toán

```

1 def suffixes(x):
2     m = len(x)
3     suff = [0] * m
4     suff[m - 1] = m
5     g = m - 1
6     f = 0
7     for i in range(m - 2, -1, -1):
8         if i > g and suff[i + m - 1 - f] < i - g:
9             suff[i] = suff[i + m - 1 - f]
10        else:
11            if i < g:
12                g = i
13            f = i
14            while g >= 0 and x[g] == x[g + m - 1 - f]:
15                g -= 1
16            suff[i] = f - g
17    return suff
18
19 def pre_bm_gs(x):

```

```

20     m = len(x)
21     suff = suffixes(x)
22     bm_gs = [m] * m
23     j = 0
24     for i in range(m - 1, -1, -1):
25         if suff[i] == i + 1:
26             while j < m - 1 - i:
27                 if bm_gs[j] == m:
28                     bm_gs[j] = m - 1 - i
29                 j += 1
30     for i in range(m - 1):
31         bm_gs[m - 1 - suff[i]] = m - 1 - i
32     return bm_gs
33
34 def pre_bm_bc(x):
35     m = len(x)
36     bm_bc = {}
37     for i in range(m):
38         bm_bc[x[i]] = m - 1 - i
39     return bm_bc
40
41 def AG(x, y):
42     m, n = len(x), len(y)
43     bm_gs = pre_bm_gs(x)
44     bm_bc = pre_bm_bc(x)
45     suff = suffixes(x)
46     skip = [0] * n
47     j = 0
48     while j <= n - m:
49         i = m - 1
50         while i >= 0:
51             k = skip[i + j]
52             s = suff[i]
53             if k > 0:
54                 if k > s:
55                     i -= s
56                     break
57                 elif k < s:
58                     i -= k
59                     break
60             else:
61                 i -= s
62         else:
63             if x[i] == y[i + j]:
64                 i -= 1
65             else:
66                 break
67     if i < 0:
68         print(j)
69     skip[j + m - 1] = m

```

```
70         shift = bm_gs[0]
71     else:
72         skip[i + j] = m - 1 - i
73         bc_shift = bm_bc.get(y[i + j], m) - m + 1 + i
74         shift = max(bm_gs[i], bc_shift)
75     j += shift
```

Chương 4

Tìm kiếm mẫu từ vị trí xác định

4.1 Thuật toán Colussi

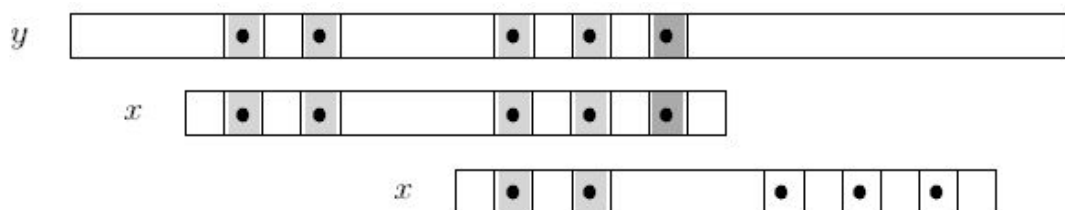
4.1.1 Trình bày thuật toán

Thuật toán Colussi là sự cải tiến của thuật toán Knuth-Morris-Pratt.

Các vị trí của mẫu (pattern) được chia làm 2 tập con. Các vị trí trong tập con thứ nhất được duyệt từ trái sang phải và các vị trí trong tập con thứ 2 (các vị trí còn lại) được duyệt từ phải sang trái.

Mỗi lần thử (mỗi cửa sổ của văn bản y) gồm 2 pha (pha 1 luôn có, pha hai có thể có hoặc không):

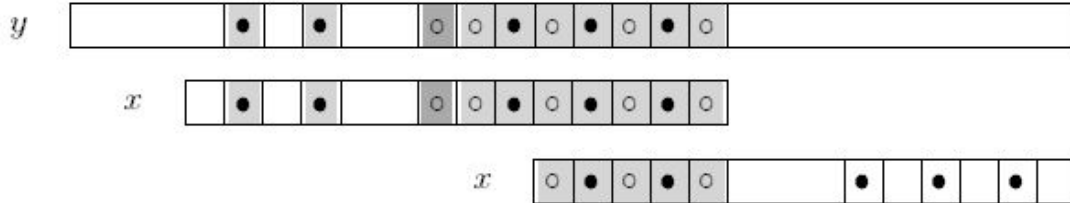
- **Pha 1** thực hiện so sánh từ trái sang phải các ký tự trong text với các ký tự trong pattern mà `kmpNext` có giá trị > -1 . Những vị trí này trong pattern được gọi là **noholes**.



Hình 4.1: Pha 1: khi dịch cửa sổ, 2 vị trí noholes (hình tròn đen) của pattern đã match với text nên không cần so sánh lại

- **Pha 2** sẽ được thực hiện khi tất cả các so sánh trong pha 1 đều match. Nếu có 1 sự không khớp xảy ra trong pha 1 thuật toán sẽ dịch chuyển cửa sổ mà không thực

hiện pha 2. Pha 2 so sánh các vị trí còn lại của pattern (các vị trí này gọi là **holes**) từ phải sang trái. Nếu có 1 sự không khớp xảy ra, thuật toán sẽ dịch chuyển cửa sổ sang vị trí tiếp theo.



Hình 4.2: Pha 2: tất cả noholes đã match, so sánh các holes (hình tròn trắng) từ phải sang; khi dịch cửa sổ không cần so sánh lại phần prefix đã match

Chiến lược này có 2 ưu điểm:

- Khi 1 sự không khớp xảy ra trong pha 1, sau khi dịch chuyển cửa sổ thích hợp ta không cần phải so sánh các vị trí noholes đã so sánh trong cửa sổ trước.
- Khi 1 sự không khớp (mismatch) xảy ra trong pha 2, nghĩa là có 1 đoạn suffix của pattern đã match với 1 đoạn của text. Sau khi dịch chuyển cửa sổ thích hợp, 1 prefix của pattern sẽ match với 1 suffix của pattern, mà suffix này lại match với 1 đoạn của text $\Rightarrow$ prefix cũng sẽ match với đoạn text đó $\Rightarrow$ khi dịch chuyển cửa sổ, ta không cần phải so sánh đoạn text đó nữa.

4.1.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(n)$.

4.1.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

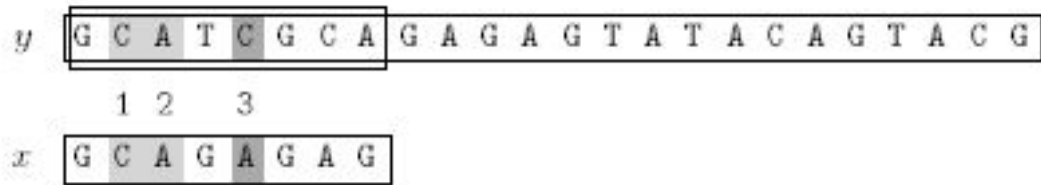
Pha tiền xử lý:

Bảng các mảng tiền xử lý của Colussi với $nd = 3$ holes (vị trí i có $\text{kmpNext}[i] = -1$) và $m - nd = 5$ noholes:

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$\text{kmpNext}[i]$	-1	0	0	-1	1	-1	1	-1	1
$\text{kmin}[i]$	0	1	2	0	3	0	5	0	
$h[i]$	1	2	4	6	7	5	3	0	
$\text{next}[i]$	0	0	0	0	0	0	0	0	0
$\text{shift}[i]$	1	2	3	5	8	7	7	7	7
$\text{hmax}[i]$	0	1	2	4	4	6	6	8	8
$\text{rmin}[i]$	7	0	0	7	0	7	0	8	
$\text{ndh0}[i]$	0	0	1	2	2	3	3	4	

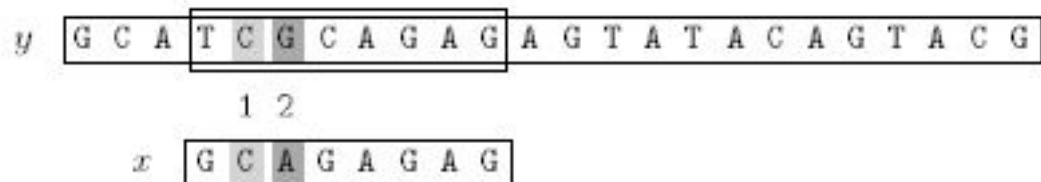
Pha tìm kiếm:

First attempt:



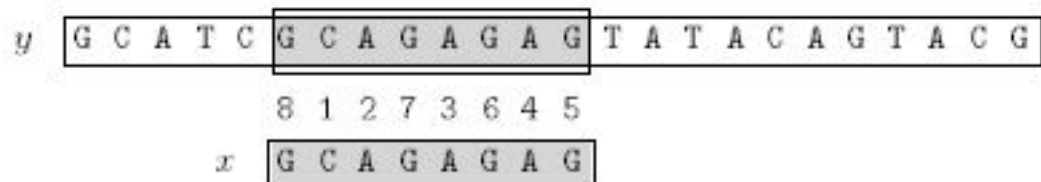
Shift by 3 ($\text{shift}[2]$)

Second attempt:



Shift by 2 ($\text{shift}[1]$)

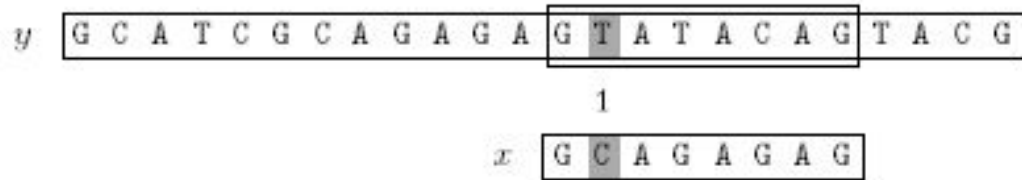
Third attempt:



Shift by 7 ($\text{shift}[8]$)

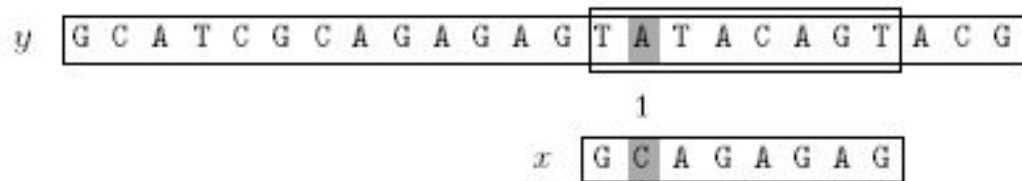
Hình 4.3: Lần 1–3: OUTPUT(5) ở lần 3

Fourth attempt:



Shift by 1 ($shift[0]$)

Fifth attempt:



Shift by 1 ($shift[0]$)

Sixth attempt:



Shift by 1 ($shift[0]$)

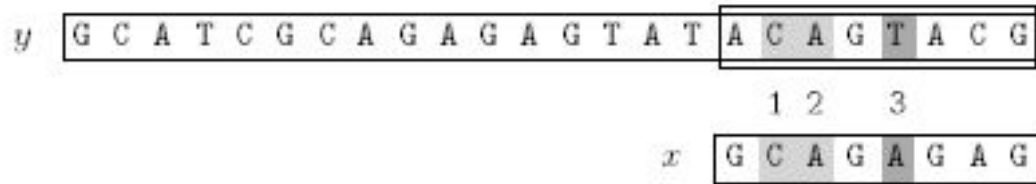
Seventh attempt:



Shift by 1 ($shift[0]$)

Hình 4.4: Lần 4–7: Dịch tiếp

Eighth attempt:



Shift by 3 (*shift*[2])

Hình 4.5: Lần 8: Kết thúc

Ta thấy thuật toán Colussi thực hiện 20 lần so sánh ký tự trong ví dụ trên.

4.1.4 Lập trình theo thuật toán

```

1 def pre_kmp(x):
2     m = len(x)
3     kmp_next = [-1] + [0] * m
4     i, j = 0, -1
5     while i < m:
6         while j > -1 and x[i] != x[j]:
7             j = kmp_next[j]
8         i += 1
9         j += 1
10        if i < m and x[i] == x[j]:
11            kmp_next[i] = kmp_next[j]
12        else:
13            kmp_next[i] = j
14    return kmp_next
15
16 def pre_colussi(x):
17     m = len(x)
18     kmp_next = pre_kmp(x)
19     hmax = [0] * (m + 1)
20     k = 1
21     while k < m:
22         q = 0
23         while q + k <= m:
24             hmax[q] = q + k
25             q += k
26         k += 1 if k == kmp_next[k] + 1 else m
27     kmin = [m] * m
28     rmin = [m] * m
29     h = [0] * m
30     s = r = 0
31     for i in range(m):

```

```

32     if hmax[i] == i + kmp_next[i + 1] + 1:
33         h[s] = i
34         s += 1
35     else:
36         h[m - 1 - r] = i
37         r += 1
38     nd = s
39     shift = [rmin[0]] * (m + 1)
40     for i in range(nd - 1, -1, -1):
41         shift[i] = kmin[h[i]]
42     nhd0 = [0] * (nd + 1)
43     for i in range(nd):
44         nhd0[i + 1] = nhd0[i] + (0 if h[i] == 0 else 1)
45     next_arr = [nhd0[shift[i]] for i in range(nd + 1)]
46     return nd, h, next_arr, shift
47
48 def COLUSSI(x, y):
49     m, n = len(x), len(y)
50     nd, h, next_arr, shift = pre_colussi(x)
51     j = last = i = 0
52     while j <= n - m:
53         while i < m and (last < j + h[i] or x[h[i]] == y[j + h[i]]):
54             i += 1
55         if i >= m:
56             print(j)
57             last = j + m - 1
58             j += shift[i]
59             i = next_arr[i]

```

4.2 Thuật toán Skip Search

4.2.1 Trình bày thuật toán

Đối với mỗi ký tự trong bảng chữ cái, thuật toán Skip Search sử dụng một thùng (bucket) để chứa tất cả các vị trí của ký tự đó trong mẫu x . Khi một ký tự xuất hiện k lần trong mẫu x , sẽ có k vị trí tương ứng trong bucket của ký tự đó. Khi chuỗi y chứa ít ký tự hơn so với bảng chữ cái, sẽ có một số bucket rỗng.

Pha tiền xử lý của thuật toán Skip Search sẽ tính bucket cho tất cả các ký tự trong bảng chữ cái.

Đối với mỗi ký tự c thuộc bảng chữ cái, ta có:

$$z[c] = \{i : 0 \leq i \leq m - 1 \text{ và } x[i] = c\}$$

$z[c]$ chính là bucket của ký tự c .

Vòng lặp chính của pha tìm kiếm sẽ kiểm tra mỗi ký tự thứ m , $y[j]$ (do đó sẽ có n/m lần lặp). Đối với mỗi $y[j]$, nó sẽ dùng mỗi vị trí trong bucket $z[y[j]]$ để kiểm tra xem liệu tại mỗi vị trí trong $z[y[j]]$ thì có tìm được một x trong y không. Nó thực hiện so sánh x với y từ vị trí đầu tiên, lần lượt tăng ký tự một cho đến khi gặp một sự không khớp hoặc tất cả đều khớp.

4.2.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m + \sigma)$.
- Pha tìm kiếm: $O(m \times n)$.

4.2.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

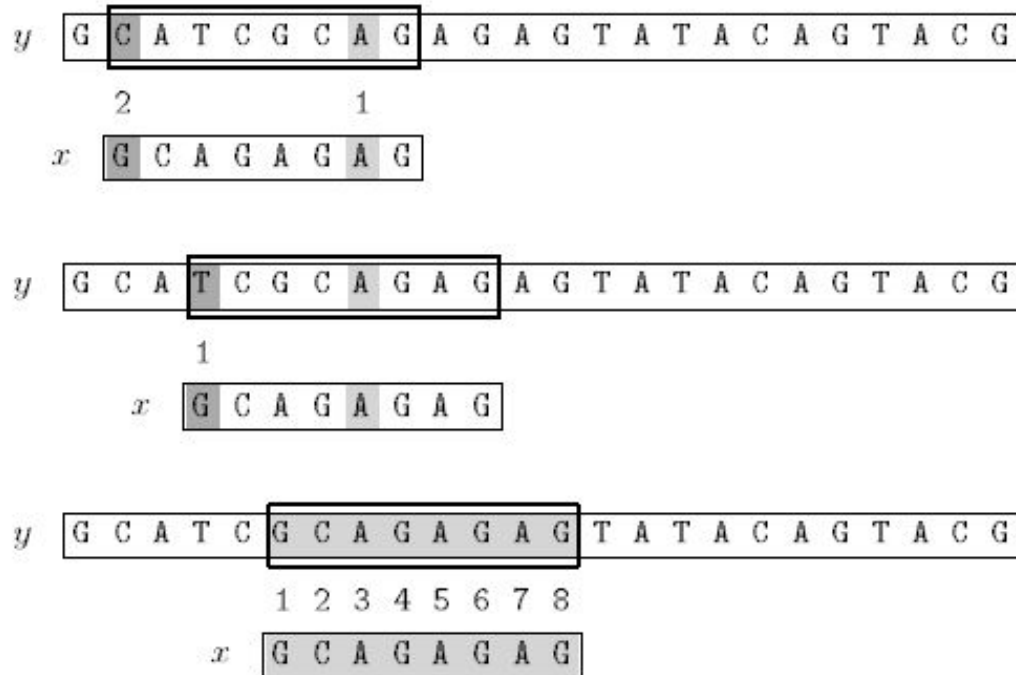
Pha tiền xử lý:

Bảng bucket $z[c]$ – tập các vị trí xuất hiện của ký tự c trong x :

c	$z[c]$
A	$\{6, 4, 2\}$
C	$\{1\}$
G	$\{7, 5, 3, 0\}$
T	$\emptyset$

Pha tìm kiếm:

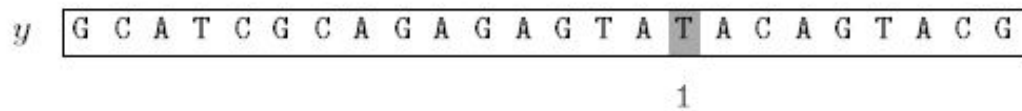
First attempt:



Shift by 8

Hình 4.6: Lần 1: OUTPUT(5)

Second attempt:



Shift by 8

Hình 4.7: Lần 2: Kết thúc

Ta thấy thuật toán Skip Search thực hiện 14 phép so sánh trong ví dụ trên.

4.2.4 Lập trình theo thuật toán

```
1 def SKIP(x, y):
2     m, n = len(x), len(y)
```

```

3  z = {}
4  for i in range(m):
5      z.setdefault(x[i], []).append(i)
6  j = m - 1
7  while j < n:
8      for i in z.get(y[j], []):
9          if j - i + m - 1 > n - 1:
10             break
11             if j - i >= 0 and y[j - i:j - i + m] == x:
12                 print(j - i)
13             j += m

```

4.3 Thuật toán Alpha Skip Search

4.3.1 Trình bày thuật toán

Là phiên bản cải tiến của thuật toán Skip Search, sử dụng các bucket để chứa các vị trí của mỗi factor độ dài $\log_{\sigma} m$.

Pha tiền xử lý của thuật toán Alpha Skip Search xây dựng một cây $T(x)$. Trong đó các nhánh của cây biểu diễn cho các factor độ dài $L = \log_{\sigma} m$ xuất hiện trong x . Sau đó, có một bucket tại mỗi lá của $T(x)$ để chứa các vị trí xuất hiện của factor (tương ứng với nhánh) trong x .

Pha tìm kiếm xem xét bucket của factor $y[j..j + L - 1]$ với tất cả $j = k \times (m - L + 1) - 1$, với k nằm trong đoạn $[1, \lfloor n/(m - L + 1) \rfloor]$.

4.3.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: $O(m)$.
- Pha tìm kiếm: $O(m \times n)$.

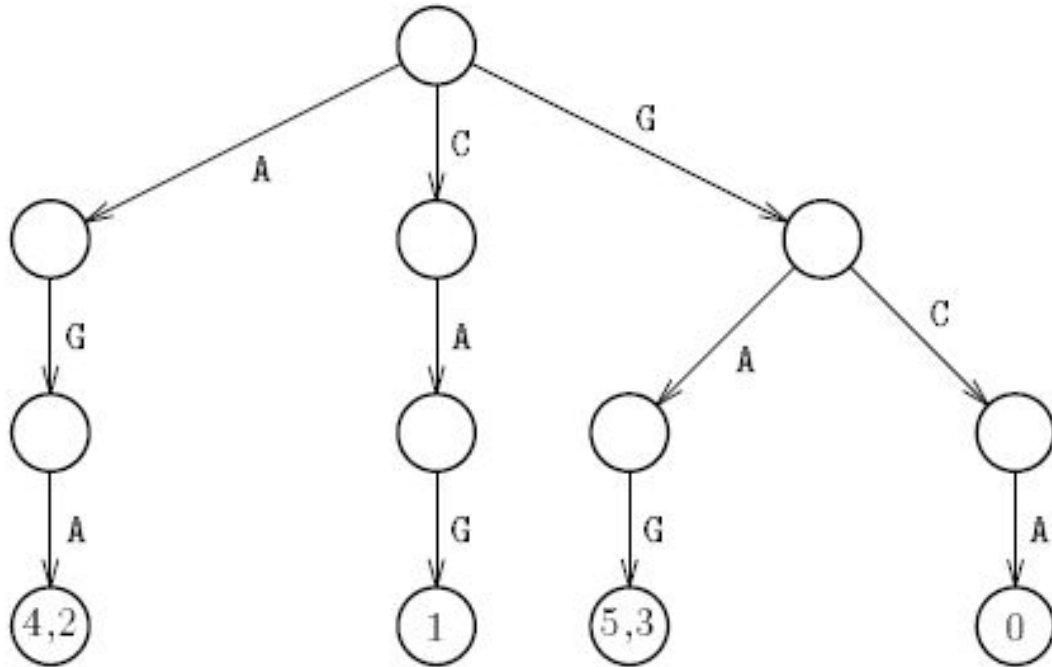
4.3.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng bucket $z[u]$ – tập vị trí xuất hiện của 3-gram u trong x :

u (3-gram)	$z[u]$
AGA	$\{4, 2\}$
CAG	$\{1\}$
GAG	$\{5, 3\}$
GCA	$\{0\}$



Hình 4.8: Trie của Alpha Skip với $q = 3$

Pha tìm kiếm:

4.3.4 Lập trình theo thuật toán

```
1 import math
2
3 def ALPHASKIP(x, y):
4     m, n = len(x), len(y)
5     sigma = 256
6     L = max(1, math.ceil(math.log(m, sigma)))
7     shift = m - L + 1
8     trie = {}
9     for i in range(m - L + 1):
10         factor = x[i:i + L]
11         trie.setdefault(factor, []).append(i)
12     j = L - 1
13     while j < n - L + 1:
14         factor = y[j:j + L]
15         for pos in trie.get(factor, []):
16             b = j - pos
17             if 0 <= b <= n - m and y[b:b + m] == x:
18                 print(b)
19     j += shift
```


Chương 5

Tìm kiếm mẫu từ vị trí bất kỳ

5.1 Thuật toán Horspool

5.1.1 Trình bày thuật toán

Thuật toán Horspool là phiên bản đơn giản hóa của thuật toán Boyer-Moore. Quy tắc dịch bad-character sử dụng trong thuật toán Boyer-Moore không hiệu quả đối với bảng chữ cái nhỏ, nhưng khi bảng chữ cái lớn so với độ dài của mẫu, như trường hợp với bảng ASCII và tìm kiếm thông thường trong một text editor thì nó trở nên rất hữu ích.

Horspool đề xuất chỉ sử dụng bad-character shift của ký tự ngoài cùng bên phải của sổ (ký tự $y[j + m - 1]$) để tính độ dịch.

5.1.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

5.1.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

Pha tiền xử lý:

Bảng Horspool bad-character shift $\text{bmBc}[a]$ – dịch theo ký tự ngoài cùng phải của cửa sổ $y[j + m - 1]$; tính theo vị trí xuất hiện phải nhất của a trong $x[0..m - 2]$:

a	A	C	G	T
$bmBc[a]$	1	6	2	8

Pha tìm kiếm:

First attempt:

y

G	C	A	T	C	G	C	A
---	---	---	---	---	---	---	---

G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 1 ($bmBc[A]$)

Second attempt:

y

G	C	A	T	C	G	C	A	G
---	---	---	---	---	---	---	---	---

A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2

1

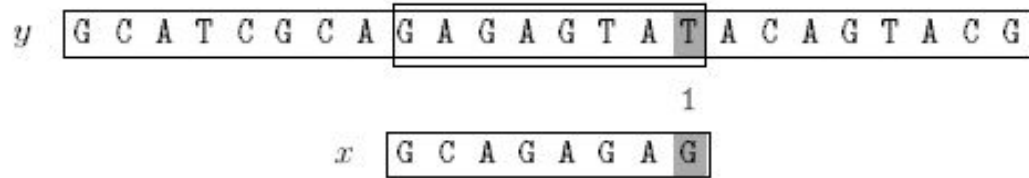
x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

Shift by 2 ($bmBc[G]$)

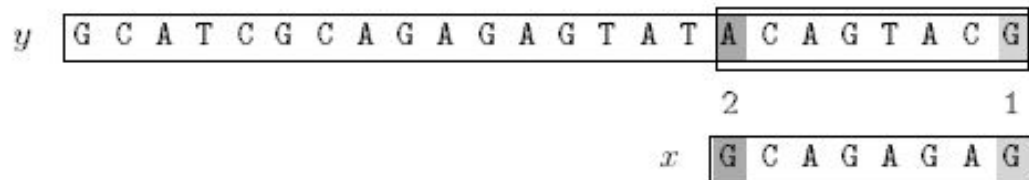
Hình 5.1: Lần 1–2: Dịch tiếp

Sixth attempt:



Shift by 8 ($bmBc[T]$)

Seventh attempt:



Shift by 2 ($bmBc[G]$)

Hình 5.3: Lần 6–7: Kết thúc

Thuật toán Horspool thực hiện 17 phép so sánh trong ví dụ trên.

5.1.4 Lập trình theo thuật toán

```

1 def pre_bm_bc(x):
2     m = len(x)
3     bm_bc = {}
4     for i in range(m):
5         bm_bc[x[i]] = m - 1 - i
6     return bm_bc
7
8 def HORSPPOOL(x, y):
9     m, n = len(x), len(y)
10    bm_bc = pre_bm_bc(x)
11    j = 0
12    while j <= n - m:
13        c = y[j + m - 1]
14        if x[m - 1] == c and x[:m - 1] == y[j:j + m - 1]:
15            print(j)
16        j += bm_bc.get(c, m)
  
```

5.2 Thuật toán Smith

5.2.1 Trình bày thuật toán

Thuật toán Smith sẽ giữ lại giá trị lớn nhất trong hai giá trị Horspool bad-character shift và Quick Search bad-character shift.

Smith nhận thấy rằng độ dịch của ký tự ngay sau ký tự ngoài cùng bên phải ($y[j + m]$) đôi khi ngắn hơn độ dịch của ký tự ngoài cùng bên phải của cửa sổ ($y[j + m - 1]$). Do đó, ông khuyên giữ lại giá trị lớn nhất trong 2 giá trị này.

Pha tiền xử lý của thuật toán Smith sẽ tính các giá trị Horspool bad-character shift và Quick Search bad-character shift.

5.2.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

5.2.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

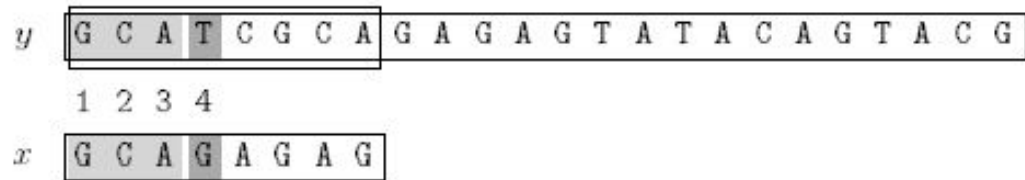
Pha tiền xử lý:

Bảng Horspool bad-character shift $\text{bmBc}[a]$ và Quick Search bad-character shift $\text{qsBc}[a]$; Smith dịch theo $\max(\text{bmBc}[y[j + m - 1]], \text{qsBc}[y[j + m]])$:

a	A	C	G	T
$\text{bmBc}[a]$	1	6	2	8
$\text{qsBc}[a]$	2	7	1	9

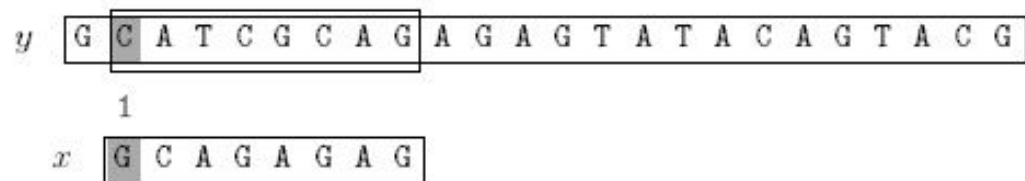
Pha tìm kiếm:

First attempt:



Shift by 1 ($bmBc[A] = qsBc[G]$)

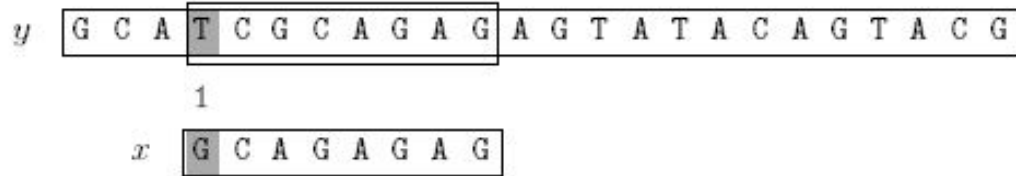
Second attempt:



Shift by 2 ($bmBc[G] = qsBc[A]$)

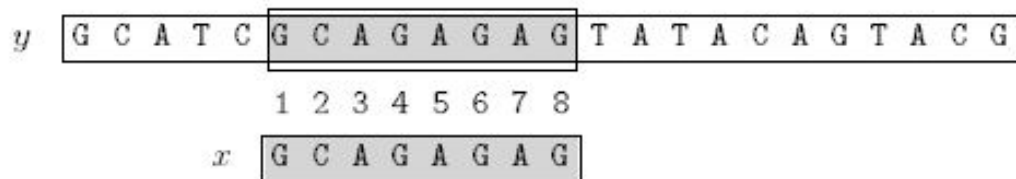
Hình 5.4: Lần 1–2: Dịch tiếp

Third attempt:



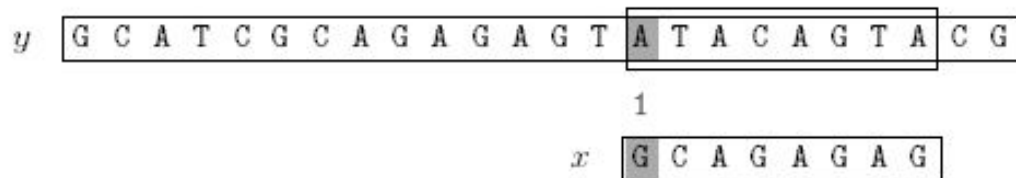
Shift by 2 ($bmBc[G] = qsBc[A]$)

Fourth attempt:



Shift by 9 ($qsBc[T]$)

Fifth attempt:



Shift by 7 ($qsBc[C]$)

Hình 5.5: Lần 3–5: OUTPUT(5) ở lần 4, Kết thúc

Thuật toán Smith thực hiện 15 phép so sánh trong ví dụ trên

5.2.4 Lập trình theo thuật toán

```

1 def pre_bm_bc(x):
2     m = len(x)
3     bm_bc = {}
4     for i in range(m):
5         bm_bc[x[i]] = m - 1 - i
6     return bm_bc
7
8 def pre_qs_bc(x):

```

```

9     m = len(x)
10    qs_bc = {}
11    for i in range(m):
12        qs_bc[x[i]] = m - i
13    return qs_bc
14
15 def SMITH(x, y):
16     m, n = len(x), len(y)
17     bm_bc = pre_bm_bc(x)
18     qs_bc = pre_qs_bc(x)
19     j = 0
20     while j <= n - m:
21         if x == y[j:j + m]:
22             print(j)
23         bc = bm_bc.get(y[j + m - 1], m)
24         qs = qs_bc.get(y[j + m], m + 1) if j + m < n else m + 1
25         j += max(bc, qs)

```

5.3 Thuật toán Raita

5.3.1 Trình bày thuật toán

Trong mỗi lần thử, đầu tiên thuật toán so sánh ký tự cuối của mẫu và ký tự cuối của cửa sổ, nếu khớp nó so sánh ký tự đầu tiên của mẫu và ký tự đầu tiên của cửa sổ, nếu khớp nó so sánh ký tự giữa của mẫu và của cửa sổ. Nếu vẫn khớp nó mới so sánh các ký tự còn lại, bắt đầu từ ký tự thứ 2. Bước dịch được tính như thuật toán Horspool.

5.3.2 Đánh giá độ phức tạp thuật toán

- Pha tiền xử lý: độ phức tạp thời gian là $O(m + \sigma)$ và độ phức tạp bộ nhớ là $O(\sigma)$.
- Pha tìm kiếm có độ phức tạp thời gian là $O(m \times n)$.

5.3.3 Kiểm nghiệm thuật toán

Với $x = \text{GCAGAGAG}$, $y = \text{GCATCGCAGAGAGTATACAGTACG}$:

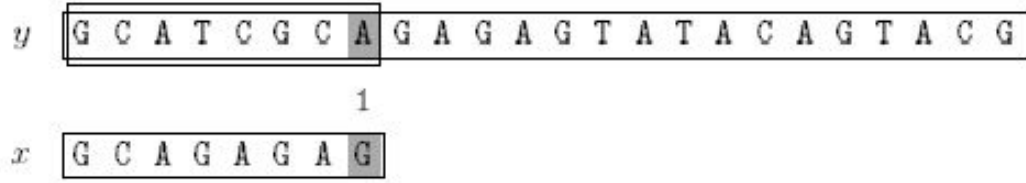
Pha tiền xử lý:

Bảng Horspool bad-character shift $\text{bmBc}[a]$ – Raita dịch theo $y[j + m - 1]$ nhưng trước đó kiểm tra $y[j]$, $y[j + m - 1]$, $y[j + m/2]$ để loại sớm:

a	A	C	G	T
$bmBc[a]$	1	6	2	8

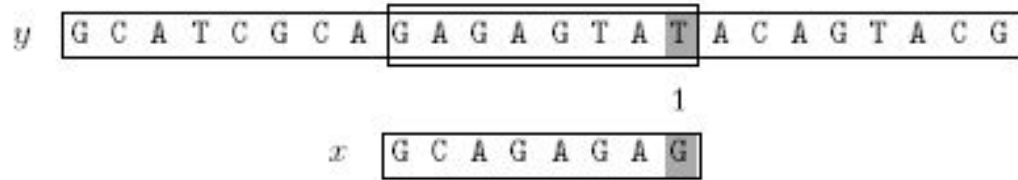
Pha tìm kiếm:

First attempt:



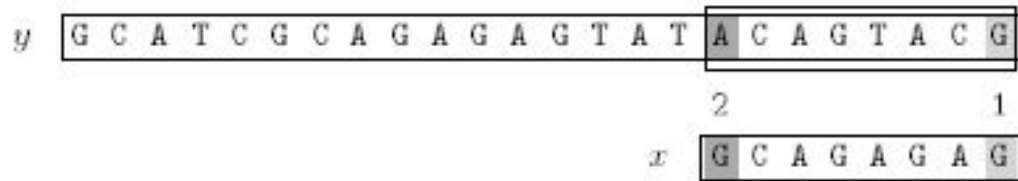
Hình 5.6: Lần 1: Kiểm tra ký tự cuối – mismatch, dịch 1 bước

Sixth attempt:



Shift by 8 ($bmBc[T]$)

Seventh attempt:



Shift by 2 ($bmBc[G]$)

Hình 5.8: Lần 6–7: Kết thúc

Ta thấy thuật toán Raita thực hiện 18 phép so sánh trong ví dụ trên.

5.3.4 Lập trình theo thuật toán

```

1 def pre_bm_bc(x):
2     m = len(x)
3     bm_bc = {}
4     for i in range(m):
5         bm_bc[x[i]] = m - 1 - i
6     return bm_bc
7
8 def RAITA(x, y):
9     m, n = len(x), len(y)
10    bm_bc = pre_bm_bc(x)
11    first, last = x[0], x[m - 1]
12    middle = x[m // 2]
13    j = 0
14    while j <= n - m:
15        if (y[j + m - 1] == last and
16            y[j] == first and
17            y[j + m // 2] == middle and
18            x[1:m - 1] == y[j + 1:j + m - 1]):
19            print(j)
20            j += bm_bc.get(y[j + m - 1], m)

```